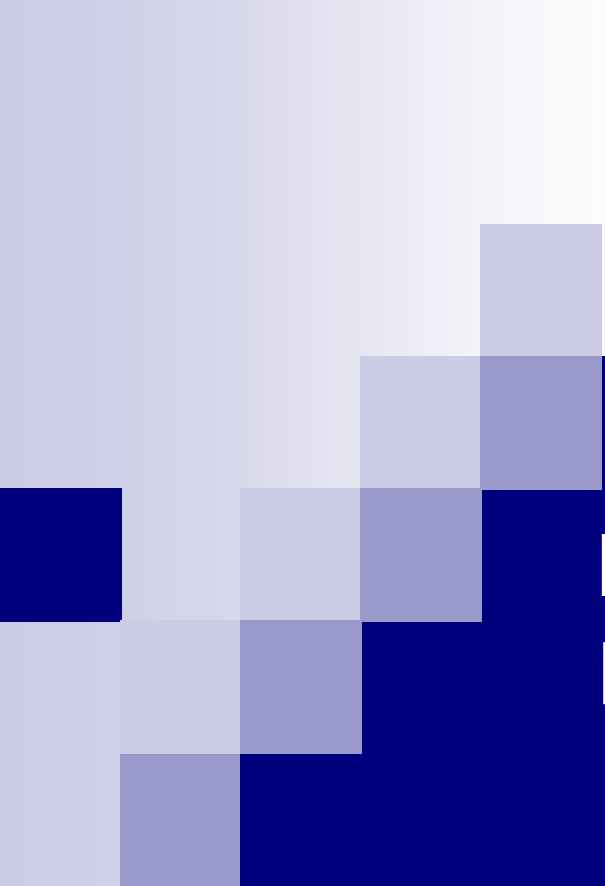




Linguaggi di Programmazione

Docente: Cataldo Musto

Capitolo 1 – Introduzione

Si ringraziano il Prof. Giovanni Semeraro, il Prof. Marco de Gemmis e il Prof. Pasquale Lops per la concessione del materiale didattico di base

Apprendimento di un linguaggio di programmazione

- Perché un insegnamento “Linguaggi di Programmazione” oltre a quello di “Programmazione”?



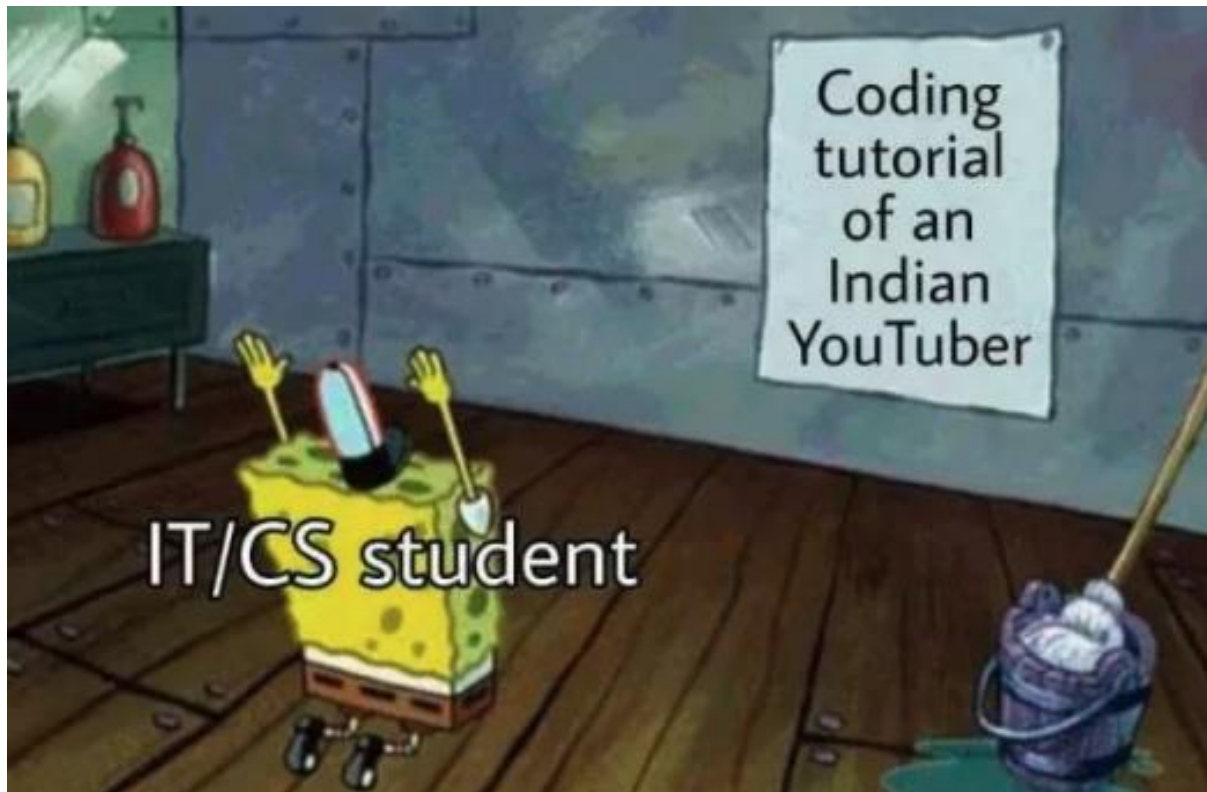


Apprendimento di un linguaggio di programmazione

- Facciamo un passo indietro
 - Come apprendere un linguaggio di programmazione?

Apprendimento di un linguaggio di programmazione

- Facciamo un passo indietro
 - Come apprendere un linguaggio di programmazione?



Apprendimento di un linguaggio di programmazione

■ Purtroppo esistono svariati linguaggi di programmazione

- è difficile conoscerne un gran numero in modo approfondito

*“According to the Online Historical Encyclopaedia of Programming Languages, people have created about **8,945 coding languages.**”*

- E' necessario **conoscere i fondamenti che sono alla base dei linguaggi di programmazione!**

Linguaggi di Programmazione vs Linguaggi Naturali

■ Lingue “naturali”

- esistono **analogie**,
somiglianze derivanti dalla
genealogia
- **Es.: italiano – rumeno**
 - cioccolata = ciocolata
 - insalata = salata
 - treno = tren
- **Es.: italiano – francese**
 - vino = vin
 - sorella - soeur

■ Linguaggi di programmazione

- esistono le stesse analogie
- è possibile conoscerne a fondo i
meccanismi che ne ispirano il
progetto e l'*implementazione*

**Lo studio degli aspetti
fondazionali dei linguaggi
di programmazione è un
passaggio chiave della
formazione universitaria e
professionale di un
informatico**

Apprendimento di un linguaggio di programmazione

- Quali **sono gli aspetti fondamentali** dei linguaggi di programmazione?
 - Gli aspetti prettamente linguistici (*e.g.*, la **sintassi** dei linguaggi)
 - Come i costrutti possono essere implementati (ed il relativo **costo**)
 - Le tecniche di **traduzione** (*e.g.*, compilazione)
 - Gli aspetti **architetturali** di un linguaggio

Apprendimento di un linguaggio di programmazione

- Quali **sono gli aspetti fondamentali** dei linguaggi di programmazione?
 - Gli aspetti prettamente linguistici (*e.g.*, la **sintassi** dei linguaggi)
 - Come i costrutti possono essere implementati (ed il relativo **costo**)
 - Le tecniche di **traduzione** (*e.g.*, compilazione)
 - Gli aspetti **architetturali** di un linguaggio

Apprendimento di un linguaggio di programmazione

- Quali **sono gli aspetti fondamentali** dei linguaggi di programmazione?
 - Gli aspetti prettamente linguistici (*e.g.*, la **sintassi** dei linguaggi)
 - Come i costrutti possono essere implementati (ed il relativo **costo**)
 - Le tecniche di **traduzione** (*e.g.*, compilazione)
 - Gli aspetti **architetturali** di un linguaggio
- Studieremo anche gli **strumenti per il riconoscimento e la generazione** dei linguaggi



Linguaggi Formali

Per poter introdurre tutti i concetti alla base del corso, è necessario introdurre il concetto di **linguaggio formale**

Cosa significa linguaggio formale?

Linguaggi Formali

- Un linguaggio si **dice 'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**

Linguaggi Formali

- Un linguaggio si **dice** **'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**
 - Il linguaggio dei codici fiscali

PMLLGR93F11C983P

La costruzione dei codici fiscali segue delle regole chiare e ben definite

Linguaggi Formali

- Un linguaggio si **dice** **'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**
 - Il linguaggio delle formule matematiche o chimiche

$$[(3 + 7) * (5 * 2)] + 3$$

La costruzione delle espressioni matematiche o delle formule chimiche segue delle regole chiare e ben definite

Linguaggi Formali

- Un linguaggio si **dice** **'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**
 - I linguaggi di programmazione

```
int a = 0;  
int b = 0;  
int c = a+b;
```

La costruzione delle istruzioni che compongono un programma segue delle regole ben definite

Linguaggi Formali

- Un linguaggio si **dice 'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**
 - Il linguaggio delle scienze: la matematica, la chimica, la fisica seguono i principi dei linguaggi formali (es., la costruzione delle espressioni in matematica). **Anche la musica. Tutti i linguaggi di programmazione**

Linguaggi Formali

- Un linguaggio si **dice 'formale'** quando il processo di costruzione delle frasi e parole di un linguaggio segue **regole precise ed univoche**
- **Esempi di linguaggi formali?**
 - Il linguaggio delle scienze: la matematica, la chimica, la fisica seguono i principi dei linguaggi formali (es., la costruzione delle espressioni in matematica). **Anche la musica. Tutti i linguaggi di programmazione**
 - Quando parliamo di «frasi» e di «parole» di un linguaggio, **si usa il termine «stringhe»**

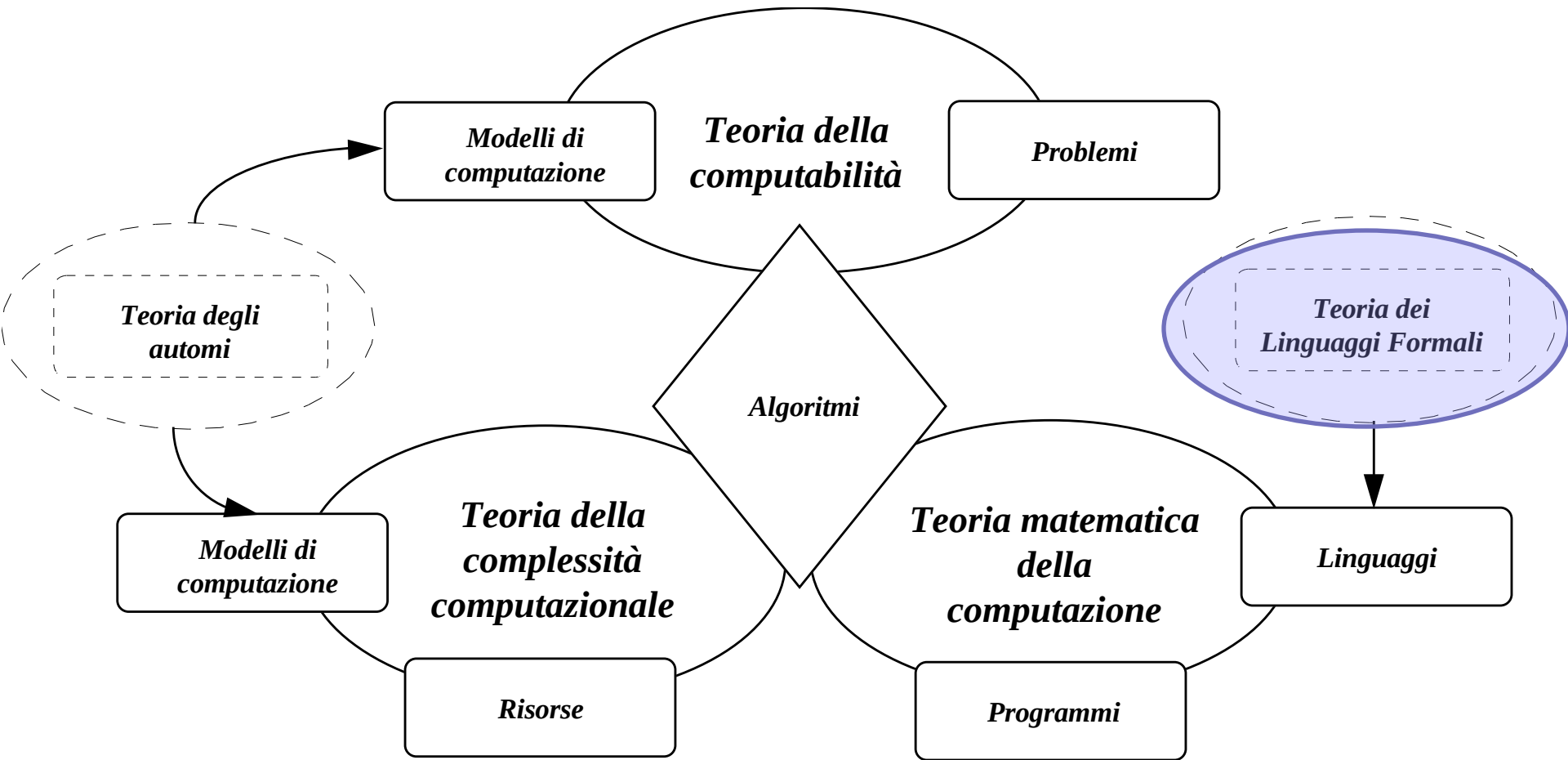
Linguaggi Formali

- Un linguaggio si **dice 'formale'** quando il processo di costruzione delle frasi e parole (*stringhe*) di un linguaggio segue **regole precise ed univoche**
- L'uso del termine **stringhe** non è casuale
 - Un codice fiscale può essere visto come una stringa
 - Una formula chimica o matematica può essere vista come una stringa
 - **Un programma può essere visto come una stringa!**
 - **In tutti questi casi possiamo definire un insieme di regole che ci dica come costruire «frasi» corrette.**

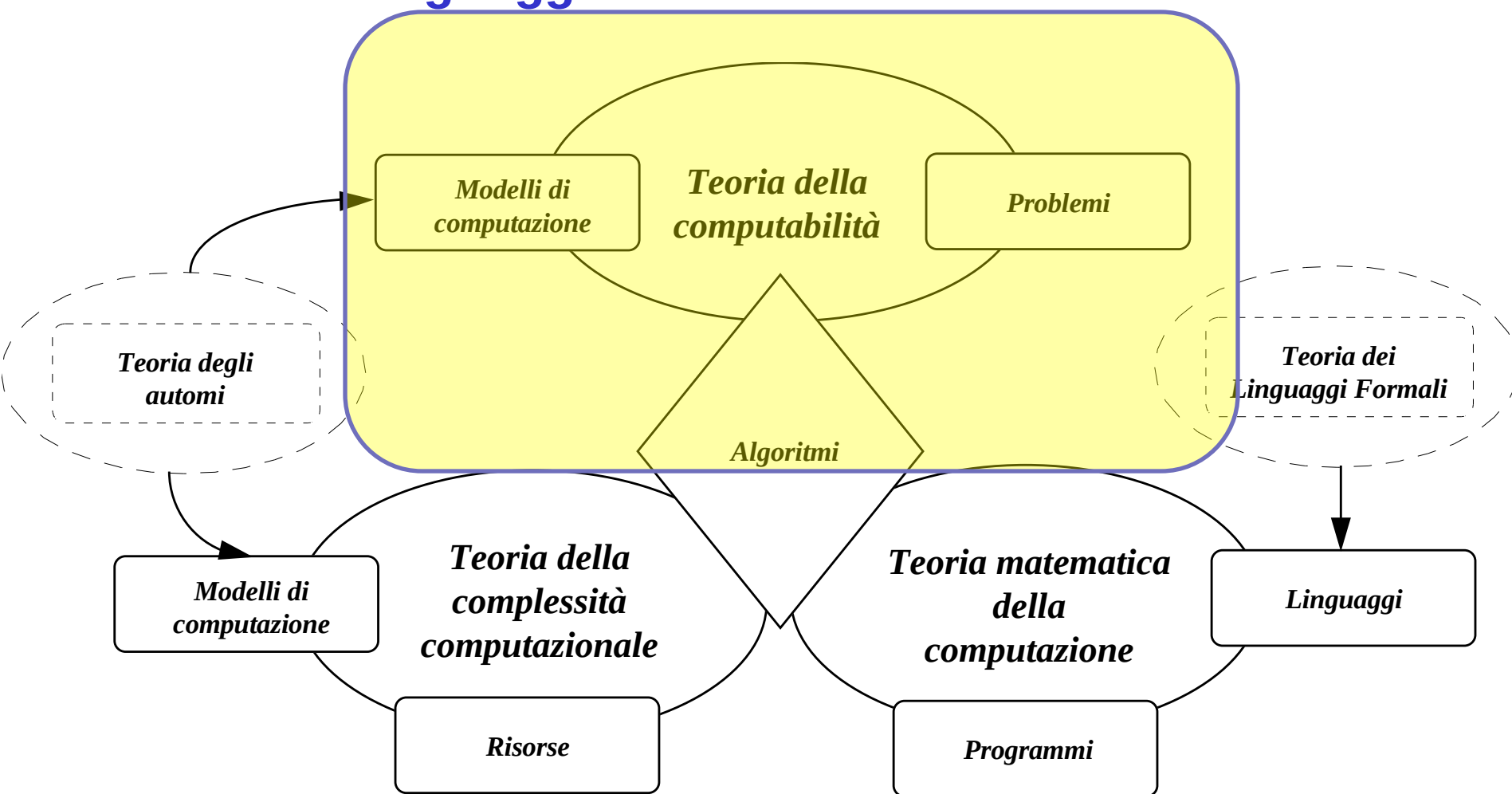
Take-home Messages (1)

- Nel corso di linguaggi di programmazione studieremo gli **aspetti fondazionali** dei linguaggi
 - Ad esempio, la **sintassi** dei linguaggi e la loro **traduzione**
- Per fare questo, ci concentreremo sui **linguaggi formali**
 - Un linguaggio formale è un linguaggio regolato da **un insieme di regole (detto 'Grammatica')**, che determina come costruire stringhe (frasi) corrette
 - **Esempi:** linguaggi di programmazione, molti linguaggi delle scienze matematiche, piccole porzioni dei linguaggi naturali
- **Domanda:** i linguaggi (e in particolar modo i linguaggi formali) sono tutti uguali o ci sono differenze tra di loro?
 - **Risposta:** ce lo dice la **teoria dei linguaggi formali**

Teoria dei Linguaggi Formali e Informatica Teorica

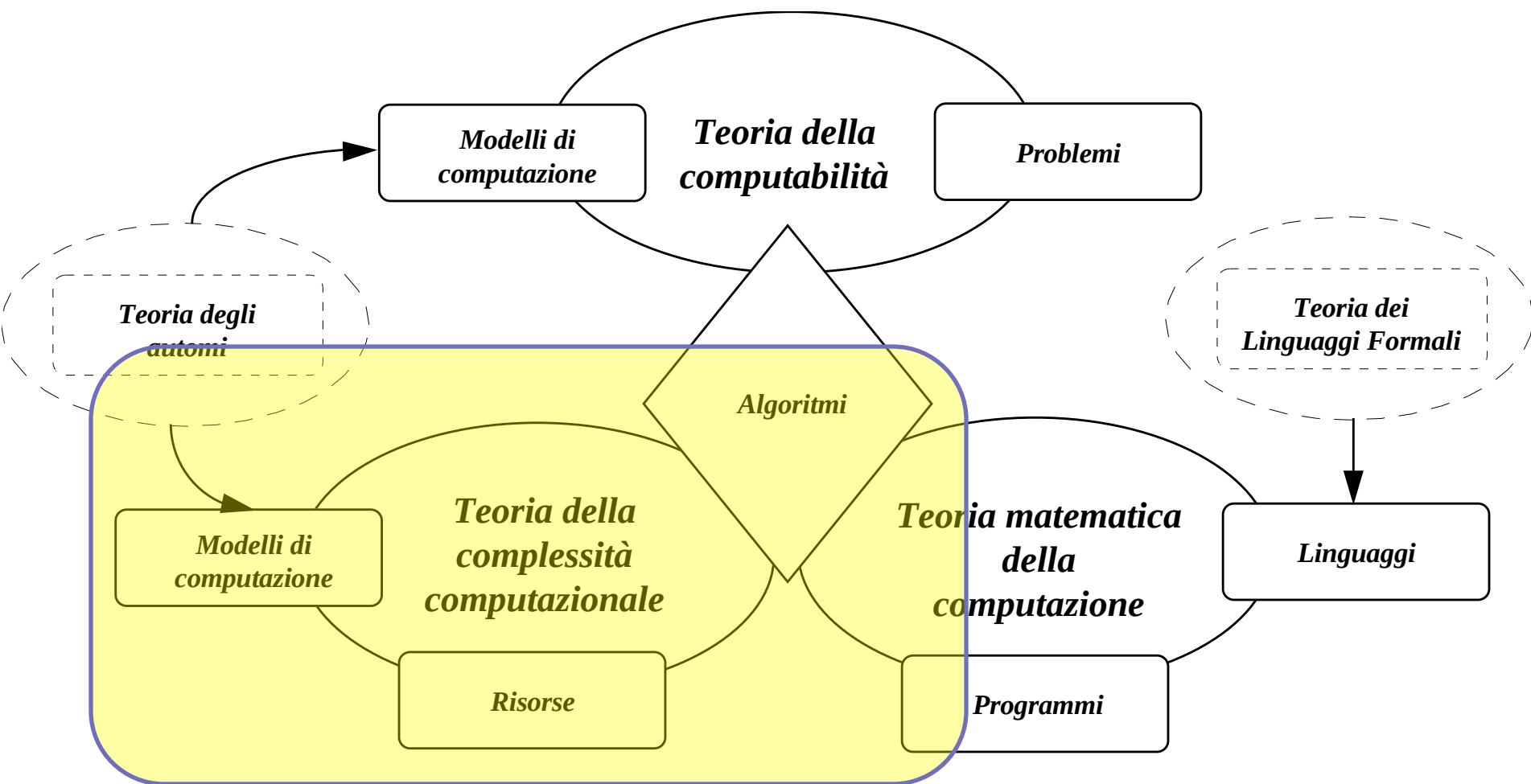


Teoria dei Linguaggi Formali e Informatica Teorica



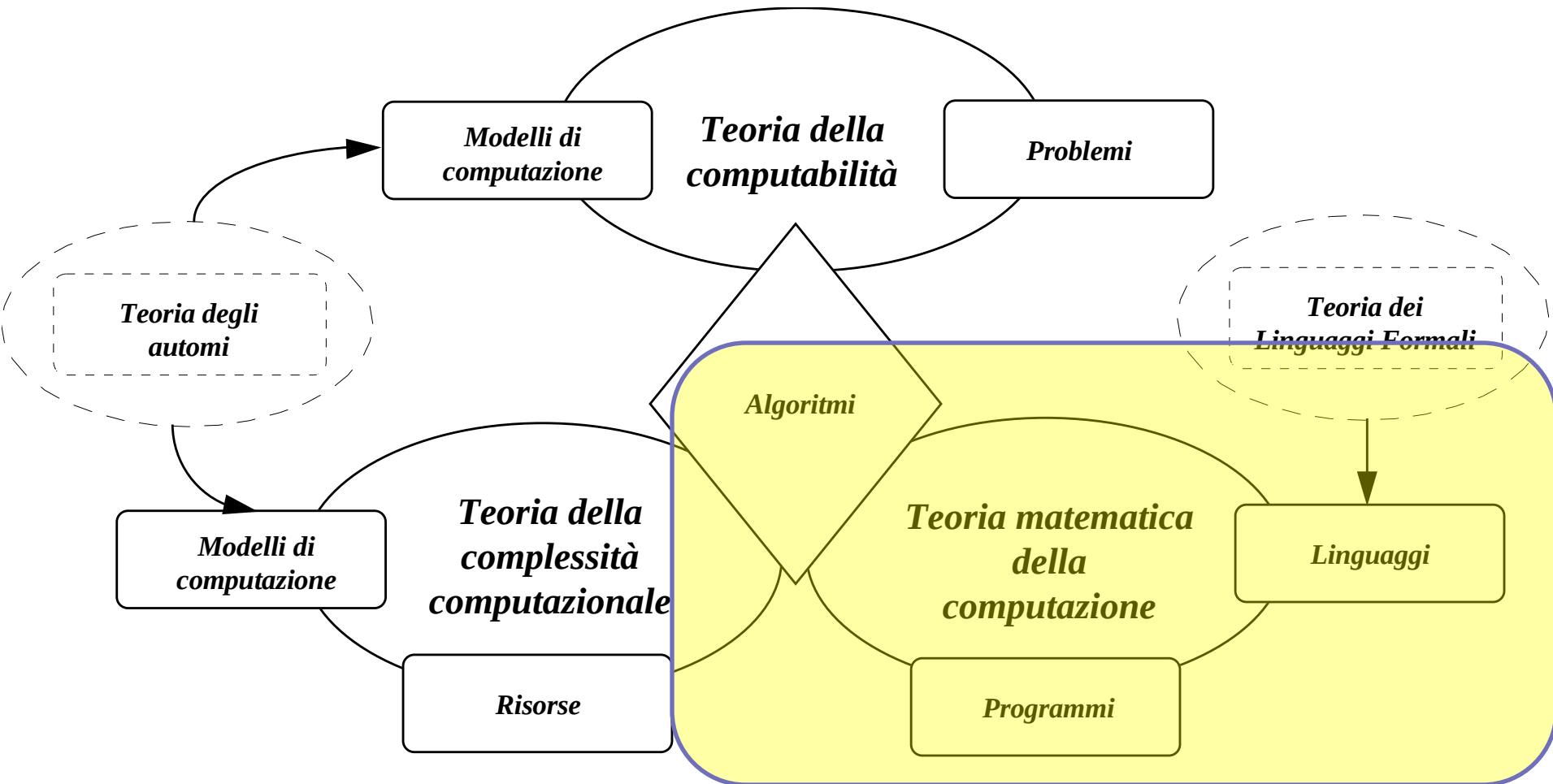
Dato un problema, **stabiliamo se esista un algoritmo** che lo risolva (lo computi).

Teoria dei Linguaggi Formali e Informatica Teorica



Dato un algoritmo, **stabiliamo se esso può essere
computato in modo efficiente.**

Teoria dei Linguaggi Formali e Informatica Teorica



Dato un algoritmo, **stabiliamo se può essere reso computabile (programma) mediante un determinato linguaggio di programmazione**

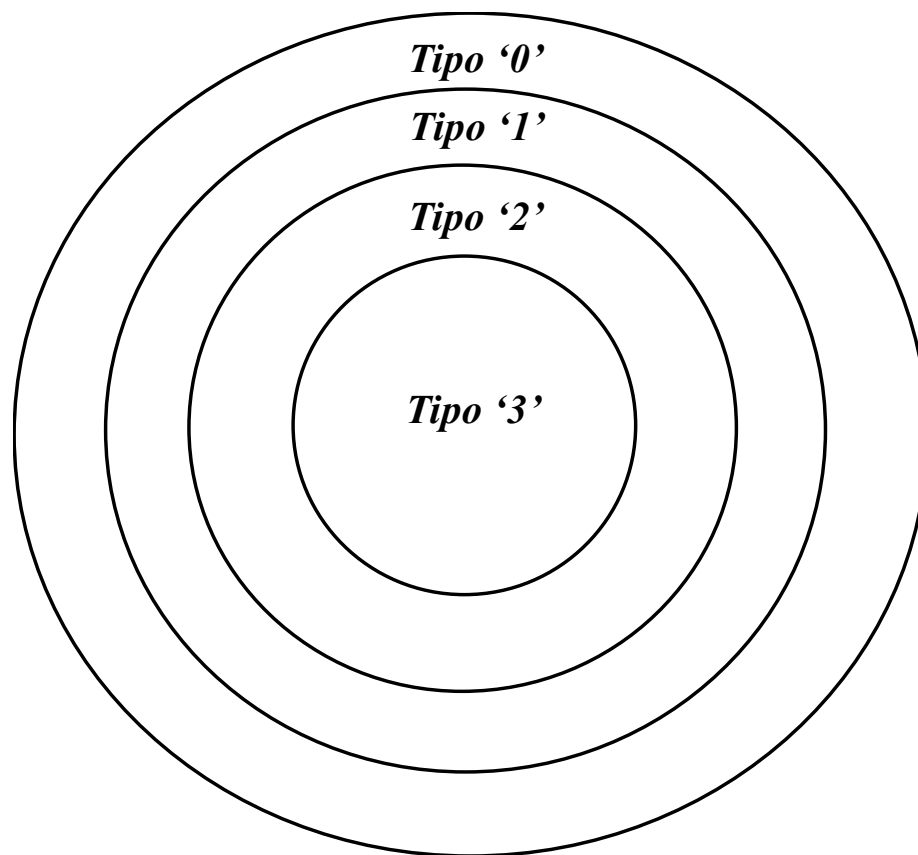
Introduzione alla teoria dei linguaggi formali

- La teoria nasce grazie al lavoro di **N. Chomsky**, linguista americano, che negli anni '50 cerca di descrivere **la sintassi del linguaggio naturale** secondo semplici **regole di riscrittura e trasformazione**.
- Chomsky classifica i linguaggi in base alle **caratteristiche delle regole** che generano tali linguaggi.



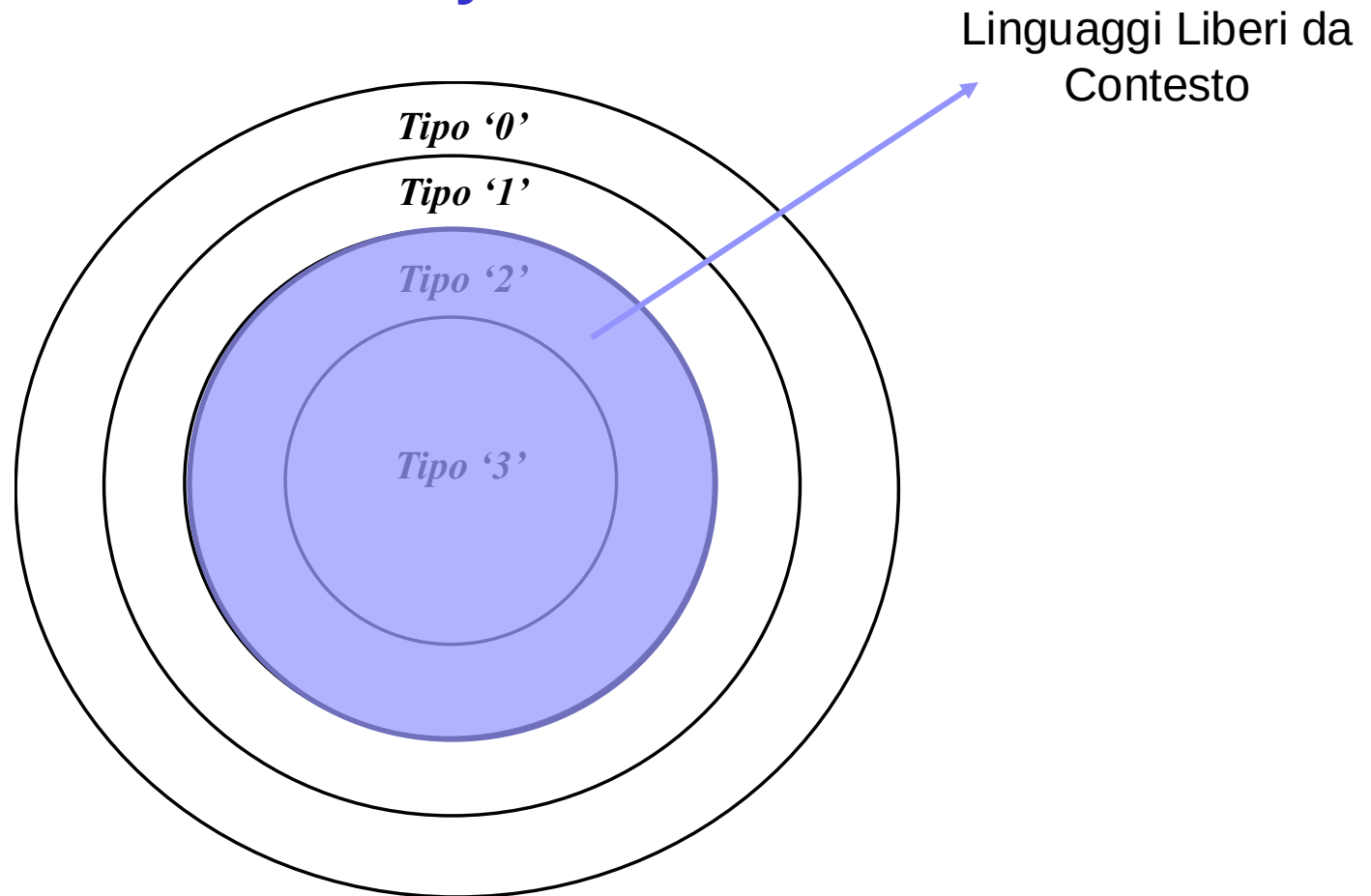
→ **La classificazione di Chomsky è uno dei fulcri del corso!**

Introduzione alla teoria dei linguaggi formali



Chomsky definisce diverse classi di linguaggi. Tipo '0' è la classe più ampia che include tutte le altre → linguaggi più complessi

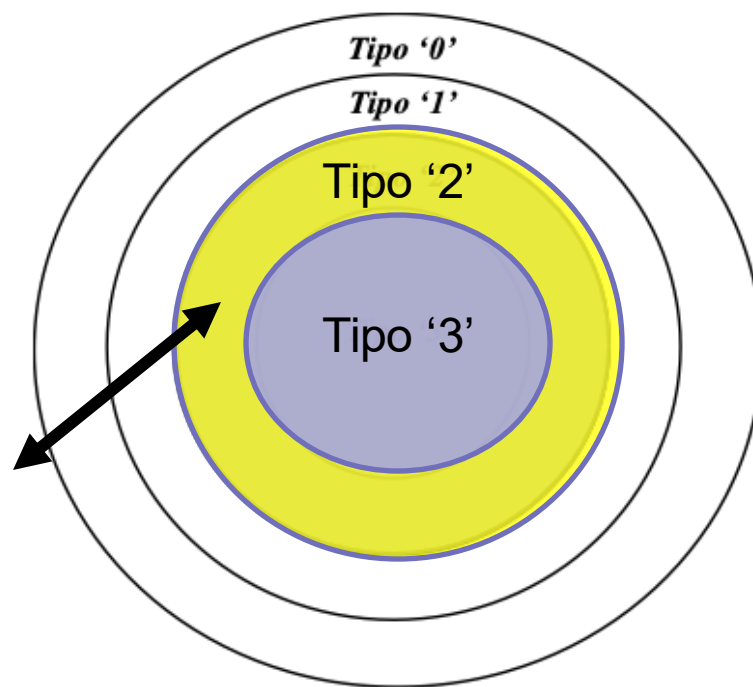
Gerarchia di Chomsky



Una classe di linguaggi molto interessante è la cosiddetta classe dei linguaggi liberi **da contesto** (privi di contesto) o **Context-free (C.F.)**, o linguaggi di tipo 2

Introduzione alla teoria dei linguaggi formali

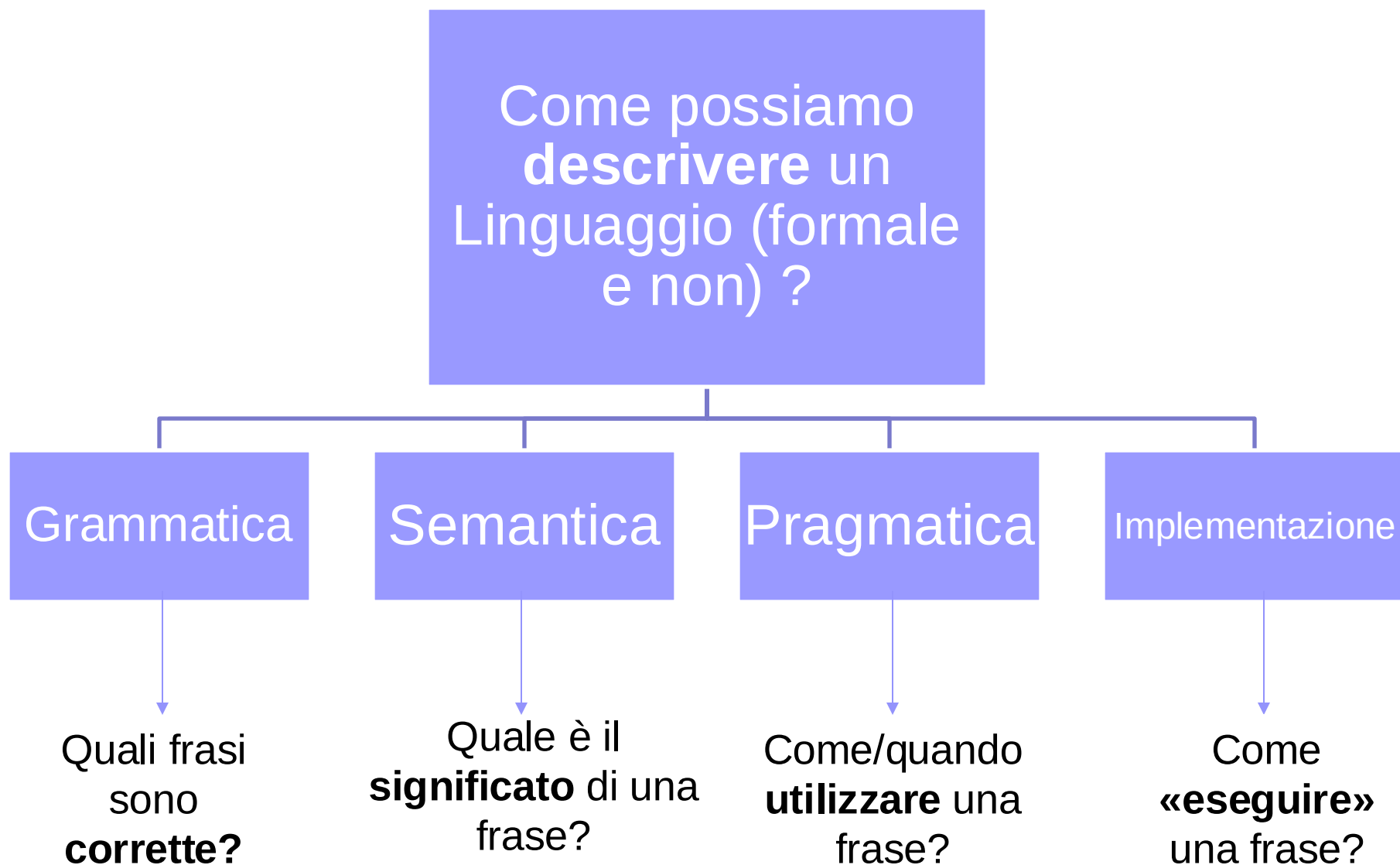
L'importanza di tale classe risiede nel fatto che lo sviluppo dei primi linguaggi di programmazione di alto livello (ALGOL 60), che segue di pochi anni il lavoro di Chomsky, **dimostra che le grammatiche C.F. sono strumenti adeguati a descrivere la sintassi di base di molti linguaggi di programmazione.**



Introduzione alla teoria dei linguaggi formali



Introduzione alla teoria dei linguaggi formali



Introduzione alla teoria dei linguaggi formali

Focus del Corso

Come possiamo
descrivere un
Linguaggio (formale
e non) ?

Grammatica

Quali frasi
sono
corrette?

Semantica

Quale è il
significato di una
frase?

Pragmatica

Come/quando
utilizzare una
frase?

Implementazione

Come
«eseguire»
una frase?

Concetto Intuitivo di grammatica

- **Le grammatiche** sono uno strumento che ci permette di determinare se e quando una stringa di un linguaggio è corretta.
- Come si definisce un grammatica?

Concetto Intuitivo di grammatica

- **Le grammatiche** sono uno strumento che ci permette di determinare se e quando una stringa di un linguaggio è corretta.
- Come si definisce un grammatica?

**complesso di regole
necessarie alla costruzione
di frasi, sintagmi e parole
(*stringhe*) di determinato
linguaggio**

Concetto Intuitivo di grammatica

■ Come si definisce un grammatica?

complesso di regole necessarie alla costruzione di frasi, sintagmi e parole (*stringhe*) di determinato linguaggio

■ Importante

- In questa definizione **non focalizzatevi troppo sui linguaggi naturali**. Le 'frasi' e le 'parole' di una lingua sono semplicemente **le stringhe valide**.
- Se considerate come linguaggio formale un linguaggio di programmazione, le stringhe valide **sono i programmi scritti correttamente**. Nel caso della chimica, formule scritte correttamente.

Concetto Intuitivo di grammatica

■ Come si definisce un grammatica?

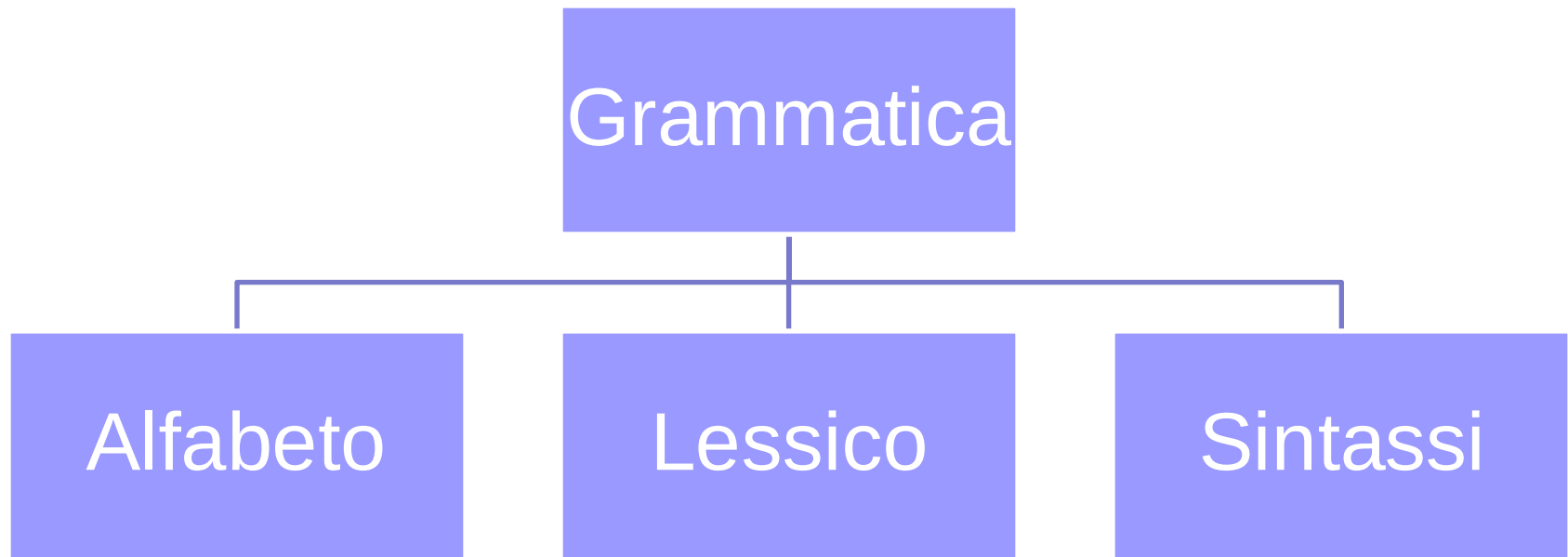
complesso di regole necessarie alla costruzione di frasi, sintagmi e parole (*stringhe*) di determinato linguaggio

■ Perché ci interessano?

- ☐ Le grammatiche sono uno strumento fondamentale per lo studio dei linguaggi di programmazione.
- ☐ Come dimostrato da Chomsky, **la sintassi di un linguaggio di programmazione si può descrivere attraverso una grammatica di un particolare tipo (tipo 2, come detto).**

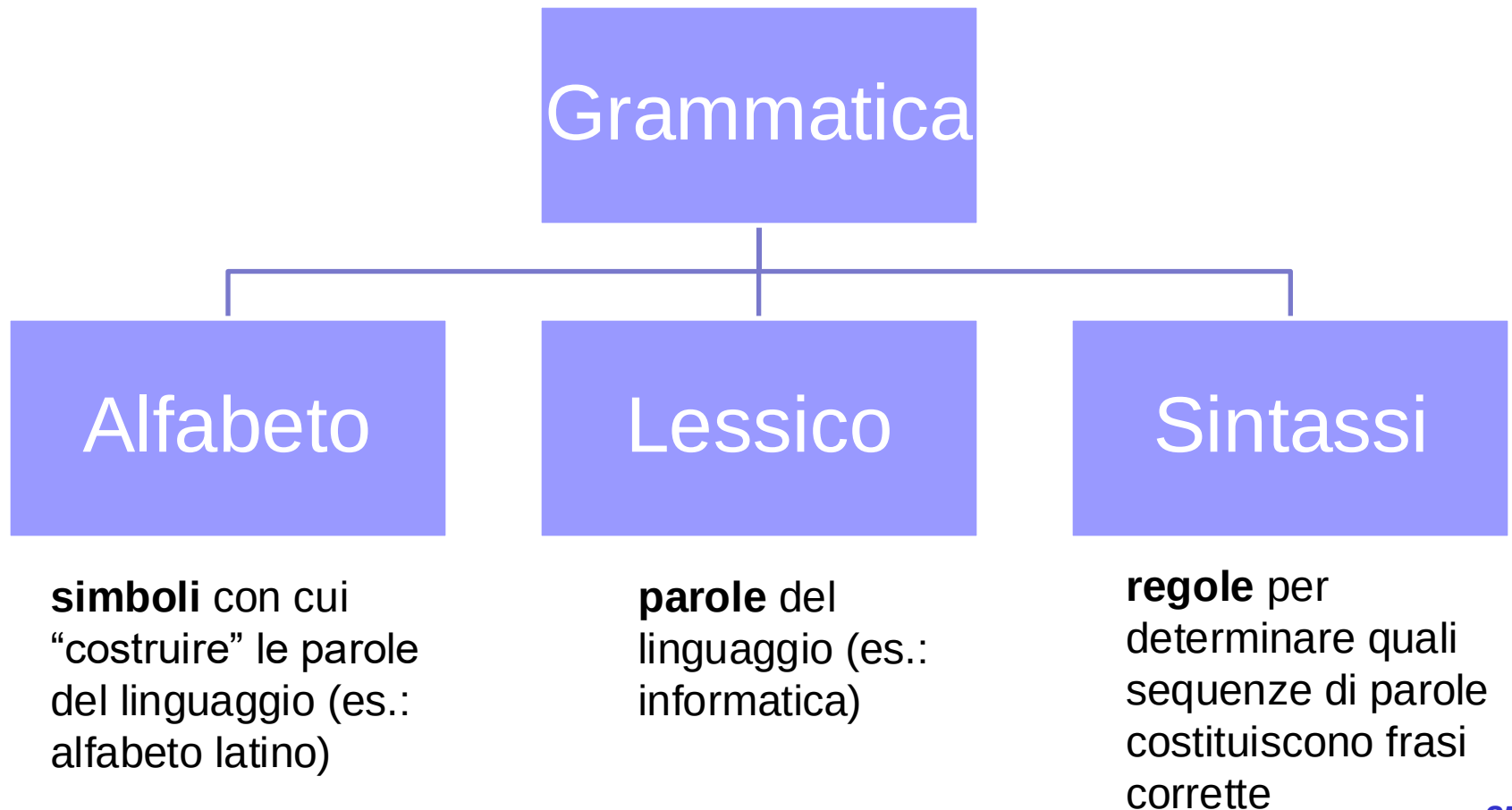
Concetto Intuitivo di grammatica

- Da quali elementi è costituita una grammatica?



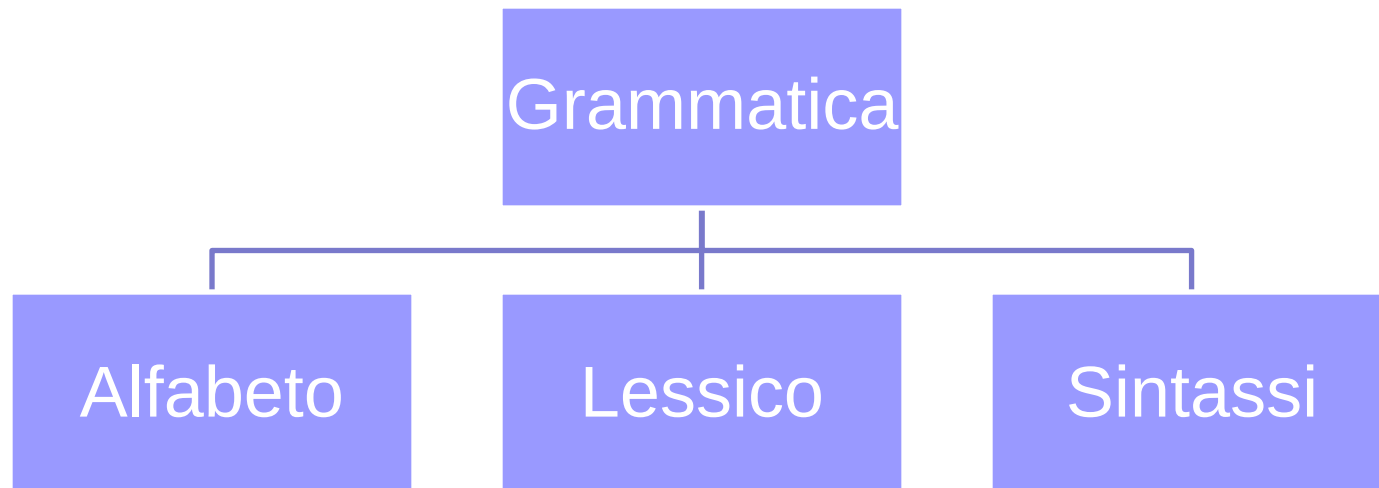
Concetto Intuitivo di grammatica

■ Da quali elementi è costituita una grammatica?



Concetto Intuitivo di grammatica

■ Da quali elementi è costituita?



■ Alfabeto, lessico e sintassi sono tra loro collegati

- Il **lessico** si costruisce capendo come utilizzare correttamente i simboli dell'**alfabeto**
- la **sintassi** si definisce capendo come combinare tra di loro correttamente **il lessico**

Concetto Intuitivo di Grammatica

- Una grammatica è un **complesso di regole**
 - Le regole di una grammatica si scrivono utilizzando la Backus-Naur Form (BNF). Ad esempio:

Regola#1 $a ::= b$

Concetto Intuitivo di Grammatica

- Una grammatica è un **complesso di regole**
 - Le regole di una grammatica si scrivono utilizzando la Backus-Naur Form (BNF). Ad esempio:

Regola#1 $a ::= b$

- In BNF, ogni regola è composta da una **parte sinistra** e una **parte destra**, inframezzate dal simbolo $::=$
- La regola si legge *'a produce b'* oppure *'a si riscrive in b'* oppure *'da a deriva b'*
- Intuitivamente una regola ci descrive come trasformare una stringa per poter ottenere **una stringa valida**
 - *"Se nella mia parola ho 'a' e la trasformo in 'b' vado nella direzione di ottenere una parola valida»"*

Concetto Intuitivo di Grammatica

- I dati anagrafici di un individuo possono essere classificati come **linguaggio formale**, perché la formulazione di una anagrafica corretta segue delle regole precise
- Supponiamo di voler definire una regola che ci dice come costruire «correttamente» una anagrafica. **Da cosa è composta?**

Concetto Intuitivo di Grammatica

- I dati anagrafici di un individuo possono essere classificati come **linguaggio formale**, perché la formulazione di una anagrafica corretta segue delle regole precise
- Supponiamo di voler definire una regola che ci dice come costruire «correttamente» una anagrafica. **Da cosa è composta?**
 - Nel nostro esempio, la regola ci può dire che l'anagrafica corretta di una persona è **composta da un nome e da un cognome**

Concetto Intuitivo di Grammatica

- Nel nostro esempio, la regola ci può dire che l'anagrafica corretta di una persona è **composta da un nome e da un cognome**

Regola#1 `<anagrafica> ::= <nome> <cognome>`



Concetto Intuitivo di Grammatica

- Nel nostro esempio, la regola ci può dire che l'anagrafica corretta di una persona è **composta da un nome e da un cognome**

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

- **Problema:** mediante questa regola ho stabilito come si costruisce una anagrafica corretta, ma non posso ancora generare delle stringhe corrette (delle coppie $\langle \text{nome}, \text{cognome} \rangle$ costruite correttamente).
- **Cosa ci manca?**
 - Suggerimento: ripensiamo agli elementi della grammatica (alfabeto, lessico, sintassi)

Concetto Intuitivo di Grammatica

- Nel nostro esempio, la regola ci può dire che l'anagrafica corretta di una persona è **composta da un nome e da un cognome**

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

- **Problema:** mediante questa regola ho stabilito come si costruisce una anagrafica corretta, ma non posso ancora generare delle stringhe corrette (delle coppie $\langle \text{nome}, \text{cognome} \rangle$ costruite correttamente).
- **Cosa ci manca?**
 - **Ho bisogno di un lessico**, cioè valori 'validi' per nome e cognome.
 - **Li codifico con altre regole**

Concetto Intuitivo di Grammatica

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi | Antonio

Regola#3 <cognome> ::= Rossi | Bianchi

Come vedete, stiamo andando nella direzione di «**costruire**» **delle regole** che ci permettano di costruire **parole** «**corrette**» in un linguaggio.

Parole = stringhe

Concetto Intuitivo di Grammatica

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi | Antonio

Regola#3 <cognome> ::= Rossi | Bianchi

- Dunque ogni regola può contenere
 - Delle categorie sintattiche (es. <nome>)
 - Il lessico del linguaggio, cioè delle **parole valide** (es. Marco).
- **Spoiler**
 - Le categorie sintattiche si chiamano **simboli non-terminali**
 - Le parole del lessico si chiamano **simboli terminali**

Concetto Intuitivo di Grammatica

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi

Regola#3 <cognome> ::= Rossi | Bianchi

- Le regole di una grammatica si chiamano anche **regole di sostituzione o riscrittura**
 - Perché applicando una regola si «trasforma» o riscrive la parte sinistra nella parte destra
 - Se c'è un pipe (simbolo '|') vuol dire che si può scegliere uno qualsiasi dei valori
 - **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- Esempio (si parte sempre dalla Regola #1)
 - $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle \quad (1)$

Quando applico una regola, la parte sinistra viene «modificata» **sostituendola con la parte destra** (si parla infatti di riscrittura)

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- Esempio (si parte sempre dalla Regola #1)
 - ☐ $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle \quad (1)$
 - ☐ $\langle \text{nome} \rangle \langle \text{cognome} \rangle ::=$
 - ☐ **Quale regola posso applicare?**

Concetto Intuitivo di Grammatica

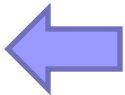
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- Esempio

☐ $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle \quad (1)$

☐ $\langle \text{nome} \rangle \langle \text{cognome} \rangle ::=$  **Applico la regola 2**

☐ Applico la regola riscrivendo il simbolo non terminale con uno dei possibili valori validi

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- Esempio

□ $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle \quad (1)$

□ $\langle \text{nome} \rangle \langle \text{cognome} \rangle ::= \text{Marco} \langle \text{cognome} \rangle \quad (2)$

Allo stesso modo, applico la regola 3

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- Esempio

□ $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$ (1)

□ $\langle \text{nome} \rangle \langle \text{cognome} \rangle ::= \text{Marco} \langle \text{cognome} \rangle$ (2)

□ $\text{Marco} \langle \text{cognome} \rangle ::= \text{Marco} \text{Rossi}$ (3)

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- L'applicazione delle regole non ha un ordine prestabilito
 - $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$ (1)
 - $\langle \text{nome} \rangle \langle \text{cognome} \rangle ::= \langle \text{nome} \rangle \text{Rossi}$ (3)
 - $\langle \text{nome} \rangle \text{Rossi} ::= \text{Marco Rossi}$ (2)

Avremmo potuto applicare anche prima la tre e poi la due, e ottenere stesso risultato

Concetto Intuitivo di Grammatica

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi

Regola#3 <cognome> ::= Rossi | Bianchi

- **Intuitivamente:** applicando le regole (in qualsiasi modo e in qualsiasi ordine possibile) riesco a ottenere delle «parole» valide del linguaggio
- L'applicazione delle regole non ha un ordine prestabilito
- **IMPORTANTE:** Un processo di applicazione delle regole prende il nome di **derivazione**

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- Il processo si può rappresentare anche con un albero, detto **albero di derivazione**

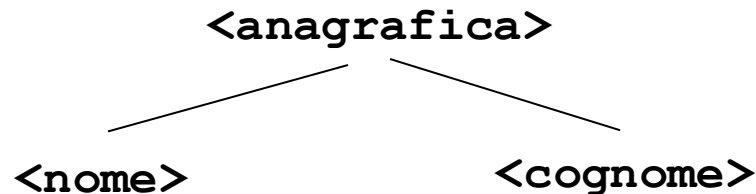
Concetto Intuitivo di Grammatica

Regola#1 `<anagrafica> ::= <nome> <cognome>`

Regola#2 `<nome> ::= Marco | Luigi`

Regola#3 `<cognome> ::= Rossi | Bianchi`

- Il processo si può rappresentare anche con un albero, detto **albero di derivazione**



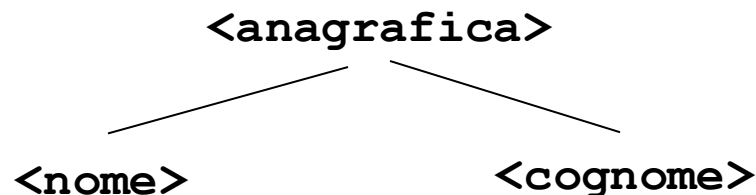
Concetto Intuitivo di Grammatica

Regola#1 `<anagrafica> ::= <nome> <cognome>`

Regola#2 `<nome> ::= Marco | Luigi`

Regola#3 `<cognome> ::= Rossi | Bianchi`

- Il processo si può rappresentare anche con un albero, detto **albero di derivazione**



- Applicando le regole, sviluppiamo ulteriori rami dell'albero

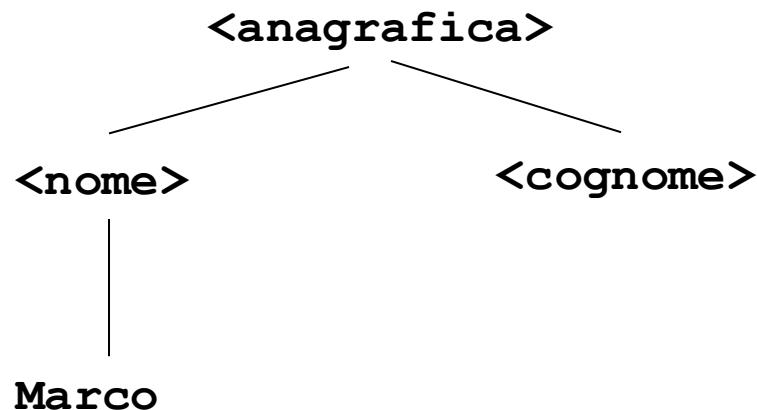
Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- Il processo si può rappresentare anche con un albero, detto **albero di derivazione**



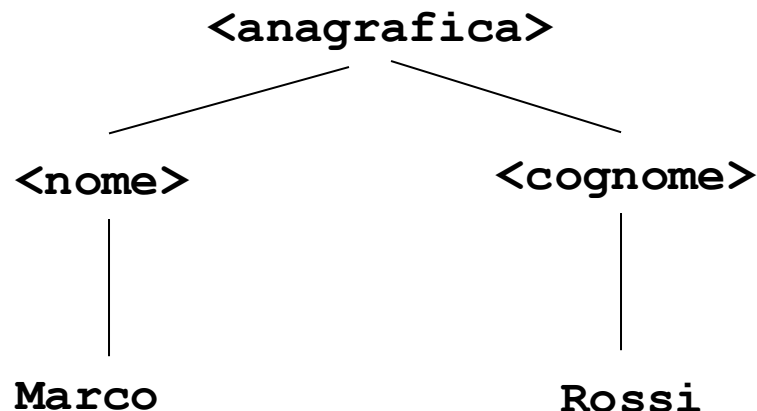
Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- Il processo si può rappresentare anche con un albero, detto **albero di derivazione**



Concetto Intuitivo di Grammatica

- **DEFINIZIONE: Correttezza** di una stringa
 - Siamo partiti dalla regola che definisce la **struttura** di una frase
 - Abbiamo applicato delle **sostituzioni** applicando le regole della grammatica
 - Siamo riusciti a **costruire** la frase da verificare usando le regole della grammatica → abbiamo **generato** la stringa

Concetto Intuitivo di Grammatica

- **DEFINIZIONE: Correttezza** di una stringa
 - Siamo partiti dalla regola che definisce la **struttura** di una frase
 - Abbiamo applicato delle **sostituzioni** applicando le regole della grammatica
 - Siamo riusciti a **costruire** la frase da verificare usando le regole della grammatica → abbiamo **generato** la stringa

CORRETTEZZA (SINTATTICA)

Una stringa è corretta se può essere generata applicando le regole della grammatica

Definizioni da imparare <3

DERIVAZIONE

il processo di generazione di una frase usando la grammatica

CORRETTEZZA (SINTATTICA)

Una frase (= sequenza di elementi del lessico) è corretta se può essere DERIVATA applicando le regole della grammatica

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

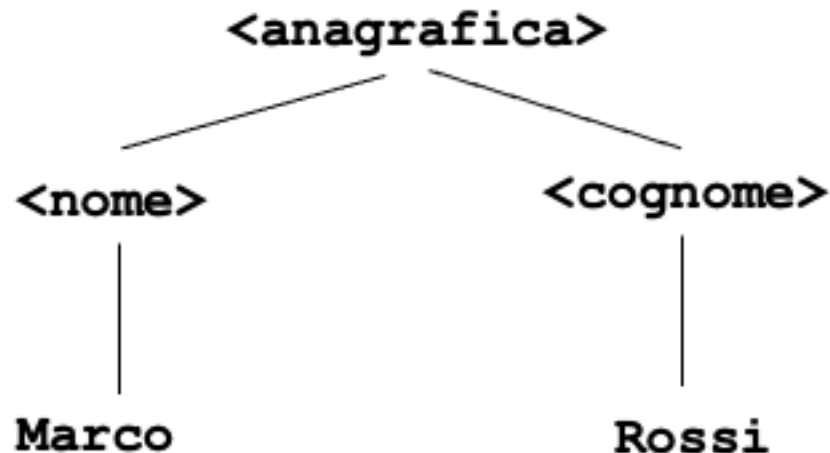
**Marco Rossi è
una stringa
corretta?**

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$



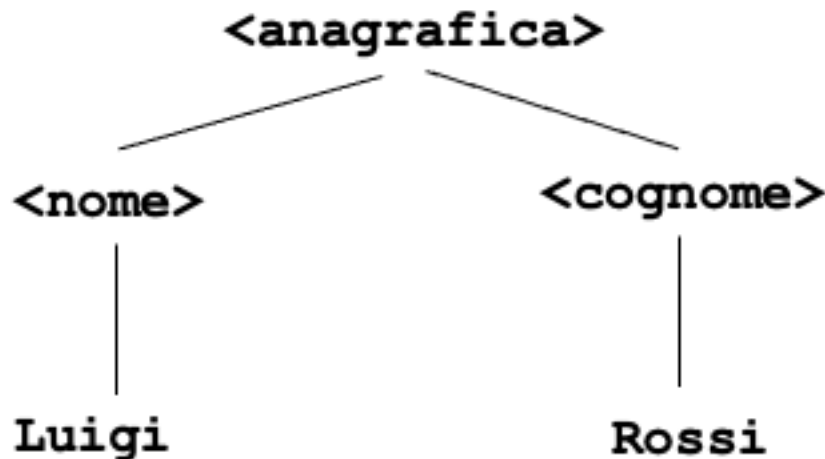
Marco Rossi è una stringa corretta? Sì, perché posso ottenerla con una sequenza di applicazione di regole.

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$



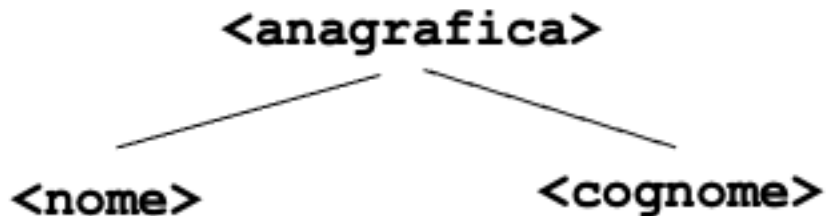
Luigi Rossi è una stringa corretta, perché posso ottenerla con una sequenza di applicazione di regole.

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$



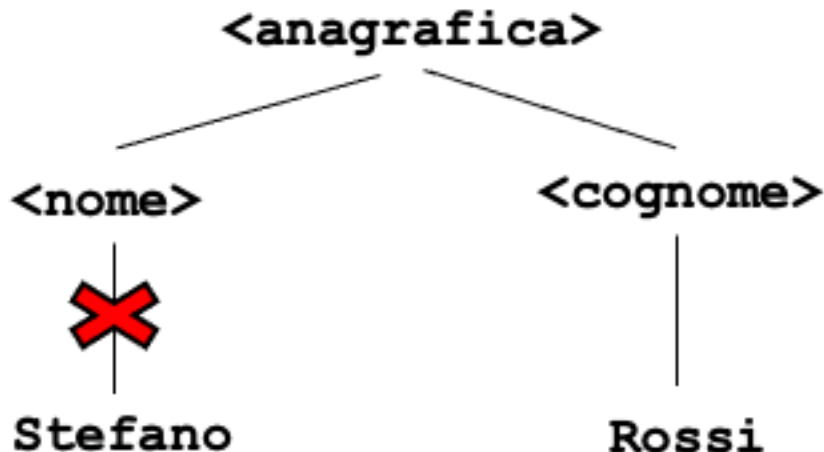
**Stefano Rossi è
è una stringa
corretta?**

Concetto Intuitivo di Grammatica

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$



**Stefano Rossi è
non è una
stringa corretta,
in questo sistema
di regole**

Esercizio

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi

Regola#3 <cognome> ::= Rossi | Bianchi

- Complichiamo un po' le cose



Puoi fermarti che mi viene da vomitare?

Esercizio

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi

Regola#3 <cognome> ::= Rossi | Bianchi

- Modifichiamo questa grammatica, prevedendo la possibilità che una persona abbia più nomi
 - ☐ Deve essere possibile derivare **Marco Luigi Rossi** oppure **Marco Antonio Bianchi**
- **Quale regola andreste a modificare, e come?**

Esercizio – Soluzione?

Regola#1 <anagrafica> ::= <nome> <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi | Antonio

Regola#3 <cognome> ::= Rossi | Bianchi

Esercizio – Soluzione?

Regola#1 `<anagrafica> ::= <nome> <nome> <cognome>`

Regola#2 `<nome> ::= Marco | Luigi | Antonio`

Regola#3 `<cognome> ::= Rossi | Bianchi`

- Apparentemente, questa grammatica è corretta, perché permette di generare coppie di nomi accanto a un cognome.
- **Ma se volessimo tre nomi?**
 - Marco Luigi Antonio Bianchi
- E se volessimo quattro nomi? :-)
 - **Dobbiamo rendere la regola più ‘generale’**

Esercizio - Soluzione

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

- Per rendere più generale la grammatica, ossia per permettere di poter derivare un numero anche infinito di nomi, **ci serve una regola di questo tipo**

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid$
 $\text{Luigi} \langle \text{nome} \rangle \mid$
 $\text{Antonio} \langle \text{nome} \rangle$

Verifichiamo la
correttezza

Esercizio - Soluzione

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco | Luigi | Antonio

Regola#3 <cognome> ::= Rossi | Bianchi

Regola#4 <nome> ::= Marco <nome> | Luigi <nome> |
Antonio <nome>

Proviamo a derivare 'Marco Luigi Rossi'

Esercizio - Soluzione

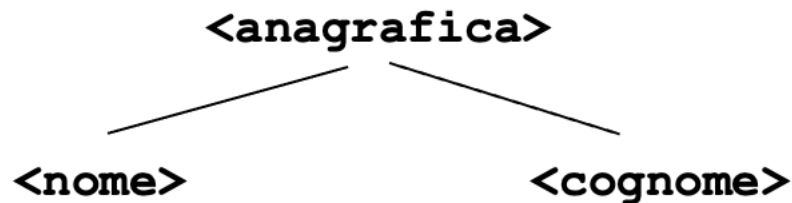
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'
Partiamo ovviamente dalla regola 1.



Cosa posso applicare a seguire?

Esercizio - Soluzione

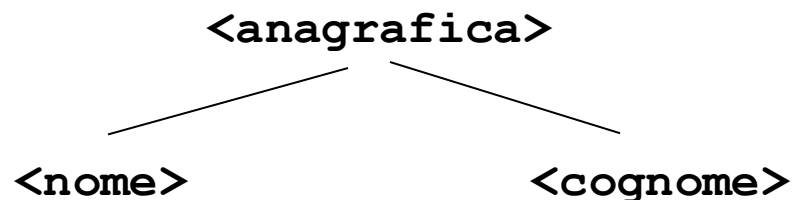
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'



Su **<nome>** posso applicare **Regola 2** e **Regola 4**
(sono quelle con **<nome>** nella parte sinistra)

Su **<cognome>** posso applicare **Regola 3**

Esercizio - Soluzione

Regola#1 <anagrafica> ::= <nome> <cognome>

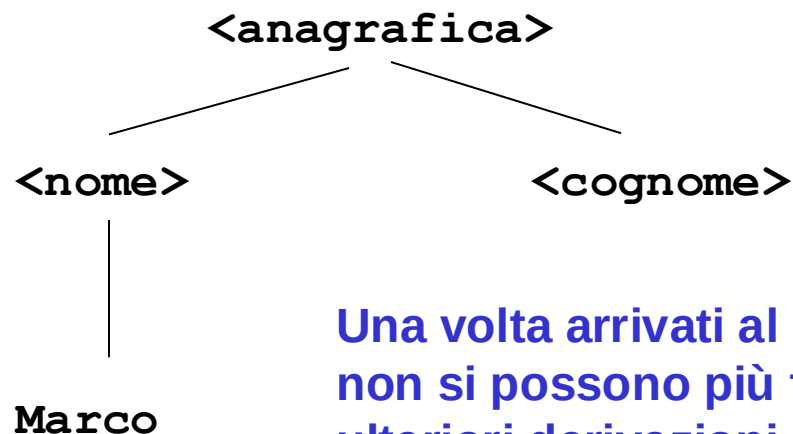
Regola#2 <nome> ::= Marco | Luigi | Antonio

Regola#3 <cognome> ::= Rossi | Bianchi

Regola#4 <nome> ::= Marco <nome> | Luigi <nome> |
Antonio <nome>

Proviamo a derivare 'Marco Luigi Rossi'

Se applicassi la
Regola#2, la
derivazione si
fermerebbe.



Una volta arrivati al lessico
non si possono più fare
ulteriori derivazioni.

Esercizio - Soluzione

Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

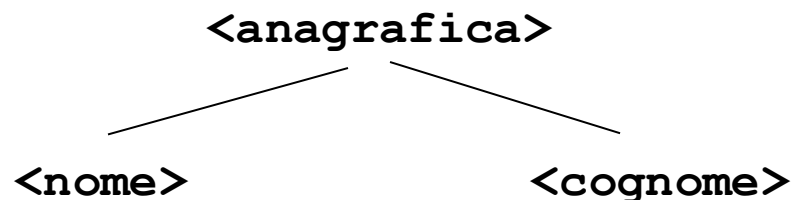
Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'

L'unica che può essere utile è a questo punto la **Regola #4**



Esercizio - Soluzione

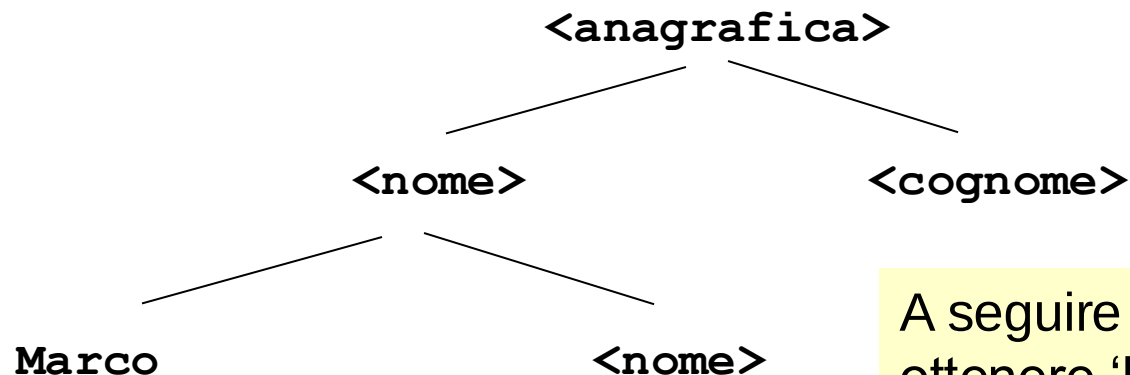
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'



A seguire mi serve ottenere 'Luigi' quindi posso applicare la **Regola #2**

Esercizio - Soluzione

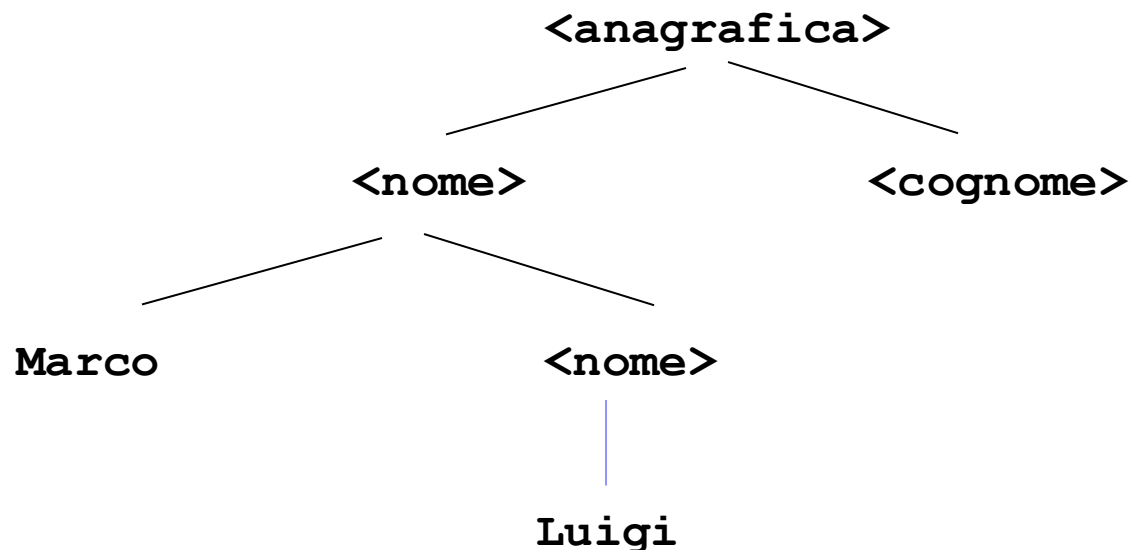
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'



Esercizio - Soluzione

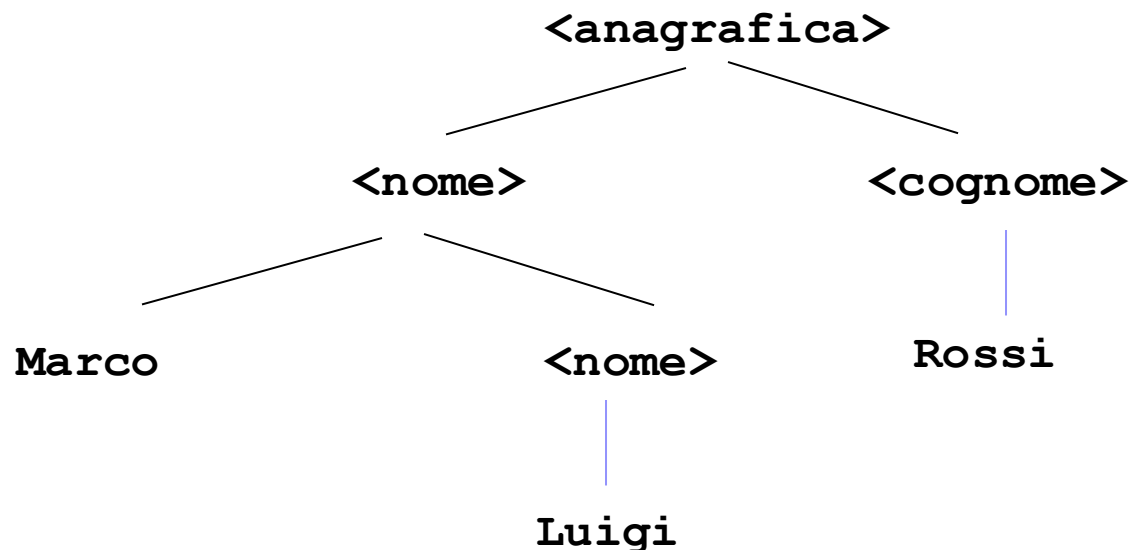
Regola#1 $\langle \text{anagrafica} \rangle ::= \langle \text{nome} \rangle \langle \text{cognome} \rangle$

Regola#2 $\langle \text{nome} \rangle ::= \text{Marco} \mid \text{Luigi} \mid \text{Antonio}$

Regola#3 $\langle \text{cognome} \rangle ::= \text{Rossi} \mid \text{Bianchi}$

Regola#4 $\langle \text{nome} \rangle ::= \text{Marco} \langle \text{nome} \rangle \mid \text{Luigi} \langle \text{nome} \rangle \mid \text{Antonio} \langle \text{nome} \rangle$

Proviamo a derivare 'Marco Luigi Rossi'



Esercizio - Soluzione

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco OR Luigi OR Antonio

Regola#3 <cognome> ::= Rossi OR Bianchi

Regola#4 <nome> ::= Marco <nome> | Luigi <nome> |
Antonio <nome>

Riusciamo a derivare anagrafiche con più di due nomi? **Provate per esercizio.**

Esercizio - Soluzione

Regola#1 <anagrafica> ::= <nome> <cognome>

Regola#2 <nome> ::= Marco OR Luigi OR Antonio

Regola#3 <cognome> ::= Rossi OR Bianchi

Regola#4 <nome> ::= Marco <nome> OR Luigi <nome> OR
Antonio <nome>

Riusciamo a derivare anagrafiche con più di due nomi? **SI**

Regole come la #4 si chiamano Regole Ricorsive, e sono di fondamentale importanza nelle grammatiche e nei linguaggi formali.

Sono regole che hanno la stessa categoria sintattica (lo stesso simbolo non-terminale) sia sulla destra che sulla sinistra, e permettono di ripetere quello specifico passo di derivazione infinite volte



Ricorsione – Breve Richiamo

- Quando una funzione è definita ricorsivamente?

Ricorsione – Breve Richiamo

- Quando una funzione è definita ricorsivamente?
- **Quando richiama sé stessa**
- Esempi?

Ricorsione – Breve Richiamo

- Quando una funzione è definita ricorsivamente?
- **Quando richiama sé stessa**
- Esempi?

- La funzione 'fattoriale'

`Fatt(n) = n!`

`Fatt(n) = 1 se n=0`

`Fatt(n) = n * Fatt(n-1) se n > 0`

Ricorsione – Breve Richiamo

- Quando una funzione è definita ricorsivamente?
- **Quando richiama sé stessa**
- Esempi?

- ☐ La funzione 'fattoriale'

`Fatt(n) = n!`

`Fatt(n) = 1 se n=0`

`Fatt(n) = n * Fatt(n-1) se n > 0`

- Analogamente, una regola ricorsiva è una regola che **richiama sé stessa più volte**

- ☐ `<nome> ::= <nome> Marco`

- ☐ Si usano quando sappiamo che una certa parola può apparire in una stringa valida, **ma non sappiamo a priori quante volte**

Concetto Intuitivo di Grammatica

- #1 $\langle \text{frase semplice} \rangle :: = \langle \text{parte nominale} \rangle \langle \text{parte verbale} \rangle$
 $\langle \text{parte nominale} \rangle$
- #2 $\langle \text{parte nominale} \rangle :: = \langle \text{articolo} \rangle \langle \text{nome} \rangle$
- #3 $\langle \text{nome} \rangle :: = \textit{car} \mid \textit{man} \mid \textit{dog}$
- #4 $\langle \text{articolo} \rangle :: = \textit{The} \mid \textit{a}$
- #5 $\langle \text{parte verbale} \rangle :: = \textit{hits} \mid \textit{eats}$

Proviamo con una Grammatica un po' più complicata

Correttezza di una frase: la grammatica

```
#1 < frase semplice > :: =< parte nominale > < parte verbale >
      < parte nominale >
#2 < parte nominale > :: =< articolo > < nome >
#3 < nome > :: = car | man | dog
#4 < articolo > :: = The | a
#5 < parte verbale > :: = hits | eats
```

Le regole #1-#2 definiscono la struttura della frase e delle categorie sintattiche

Regola#1 definisce la struttura della frase

Regola#2 definisce la struttura della **parte nominale**

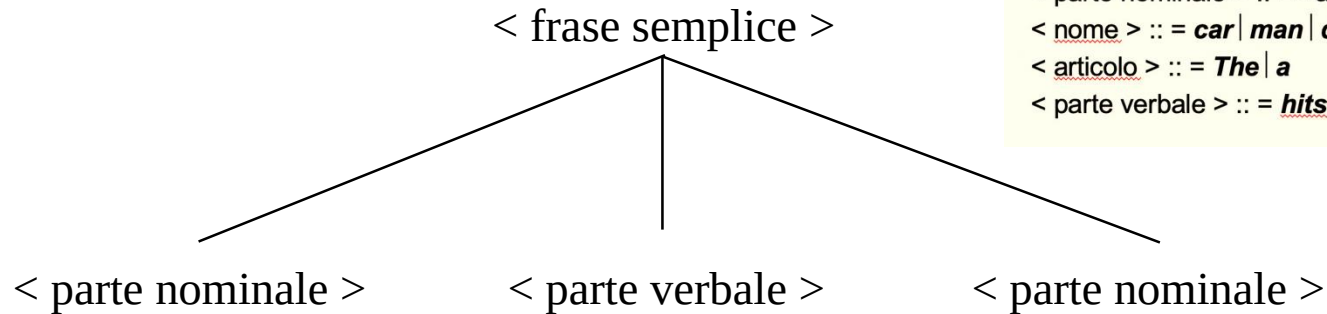
```
#1 < frase semplice > :: =< parte nominale > < parte verbale >
                                < parte nominale >
#2 < parte nominale > :: =< articolo > < nome >
#3 < nome > :: = car | man | dog
#4 < articolo > :: = The | a
#5 < parte verbale > :: = hits | eats
```

88/149

Sintassi e semantica

Provate a costruire un albero di derivazione per la frase «The Man Hits The Dog»

Albero di derivazione

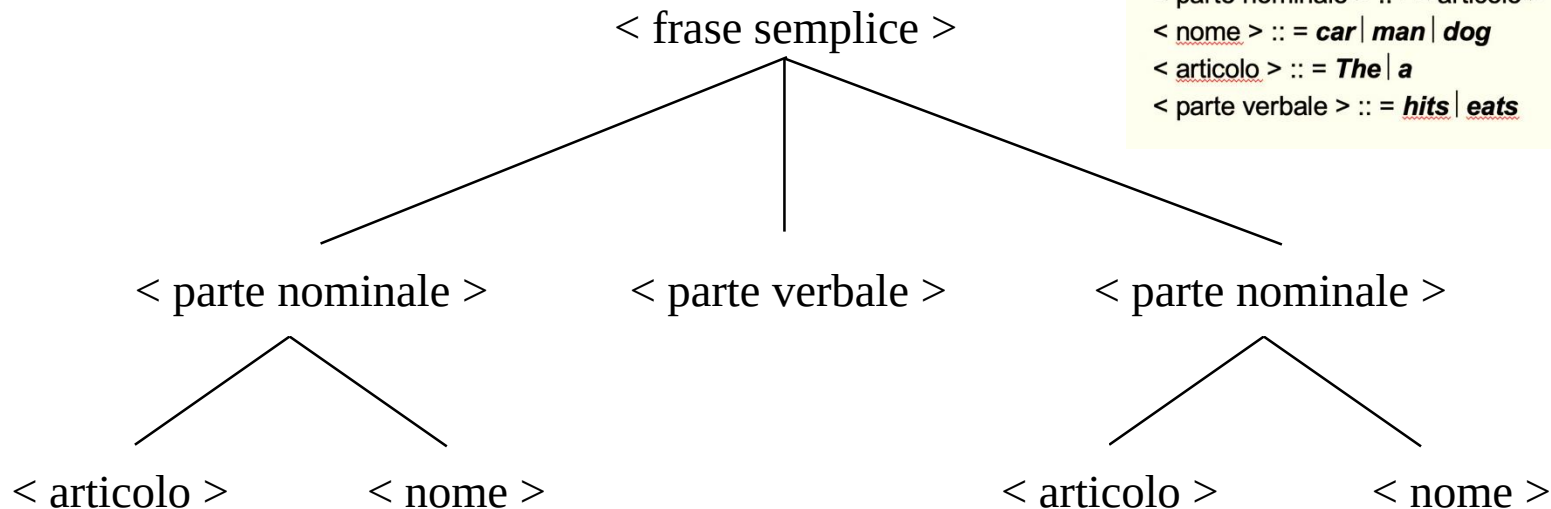


< frase semplice > :: =< parte nominale > < parte verbale >
< parte nominale >
< parte nominale > :: =< articolo > < nome >
< nome > :: = **car** | **man** | **dog**
< articolo > :: = **The** | **a**
< parte verbale > :: = **hits** | **eats**

Si parte sempre con la Regola#1

Albero di derivazione

< frase semplice > :: =< parte nominale > < parte verbale >
< parte nominale >
< parte nominale > :: =< articolo > < nome >
< nome > :: = **car** | **man** | **dog**
< articolo > :: = **The** | **a**
< parte verbale > :: = **hits** | **eats**



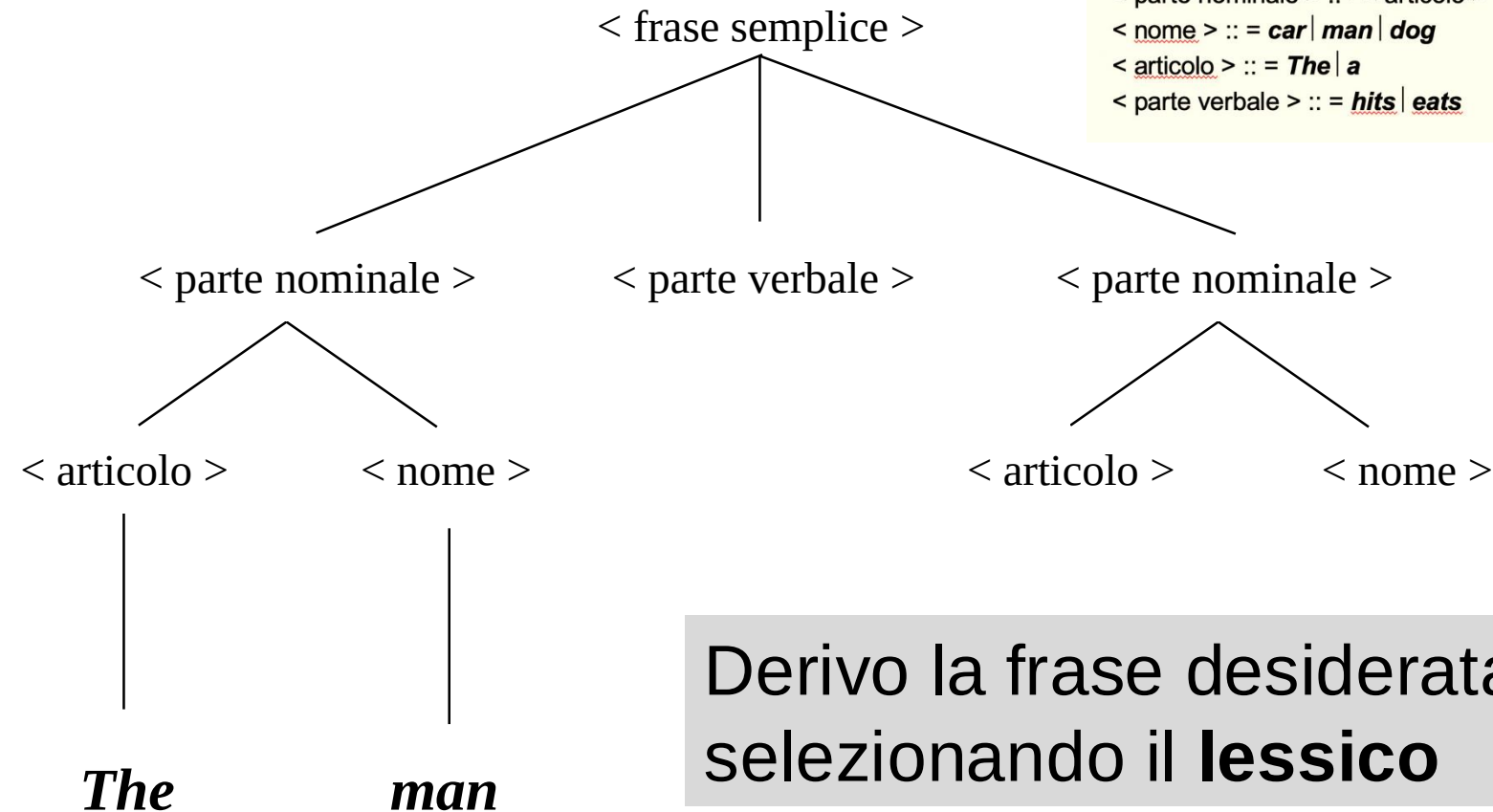
Applico la regola per sviluppare le due parti nominali

Sintassi e semantica

Provate a costruire un albero di derivazione per la frase «The Man Hits The Dog»

Albero di derivazione

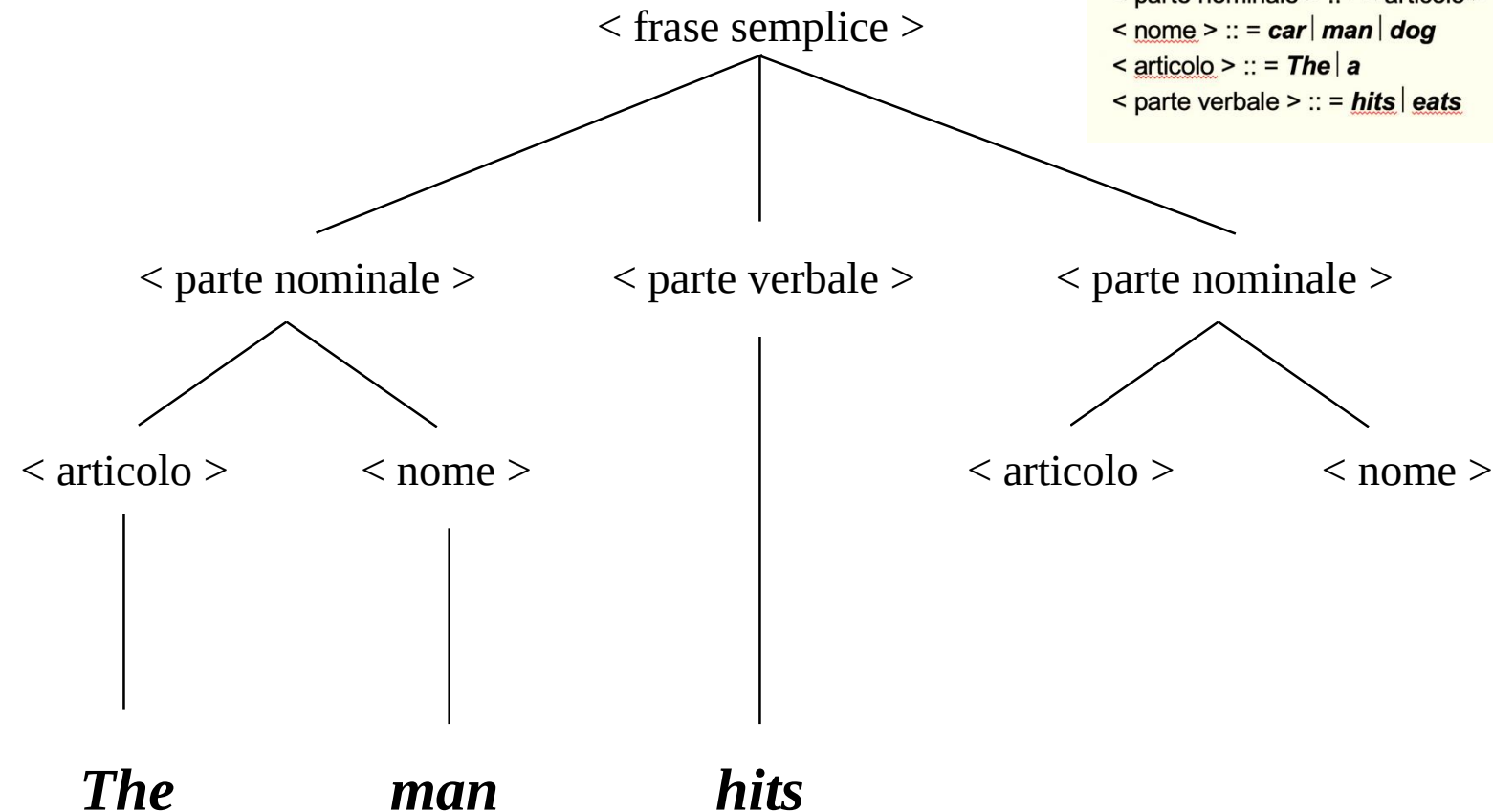
< frase semplice > :: =< parte nominale > < parte verbale >
< parte nominale >
< parte nominale > :: =< articolo > < nome >
< nome > :: = *car* | *man* | *dog*
< articolo > :: = *The* | *a*
< parte verbale > :: = *hits* | *eats*



Derivo la frase desiderata
selezionando il **lessico**

Albero di derivazione

< frase semplice > :: =< parte nominale > < parte verbale >
< parte nominale >
< parte nominale > :: =< articolo > < nome >
< nome > :: = *car* | *man* | *dog*
< articolo > :: = *The* | *a*
< parte verbale > :: = *hits* | *eats*

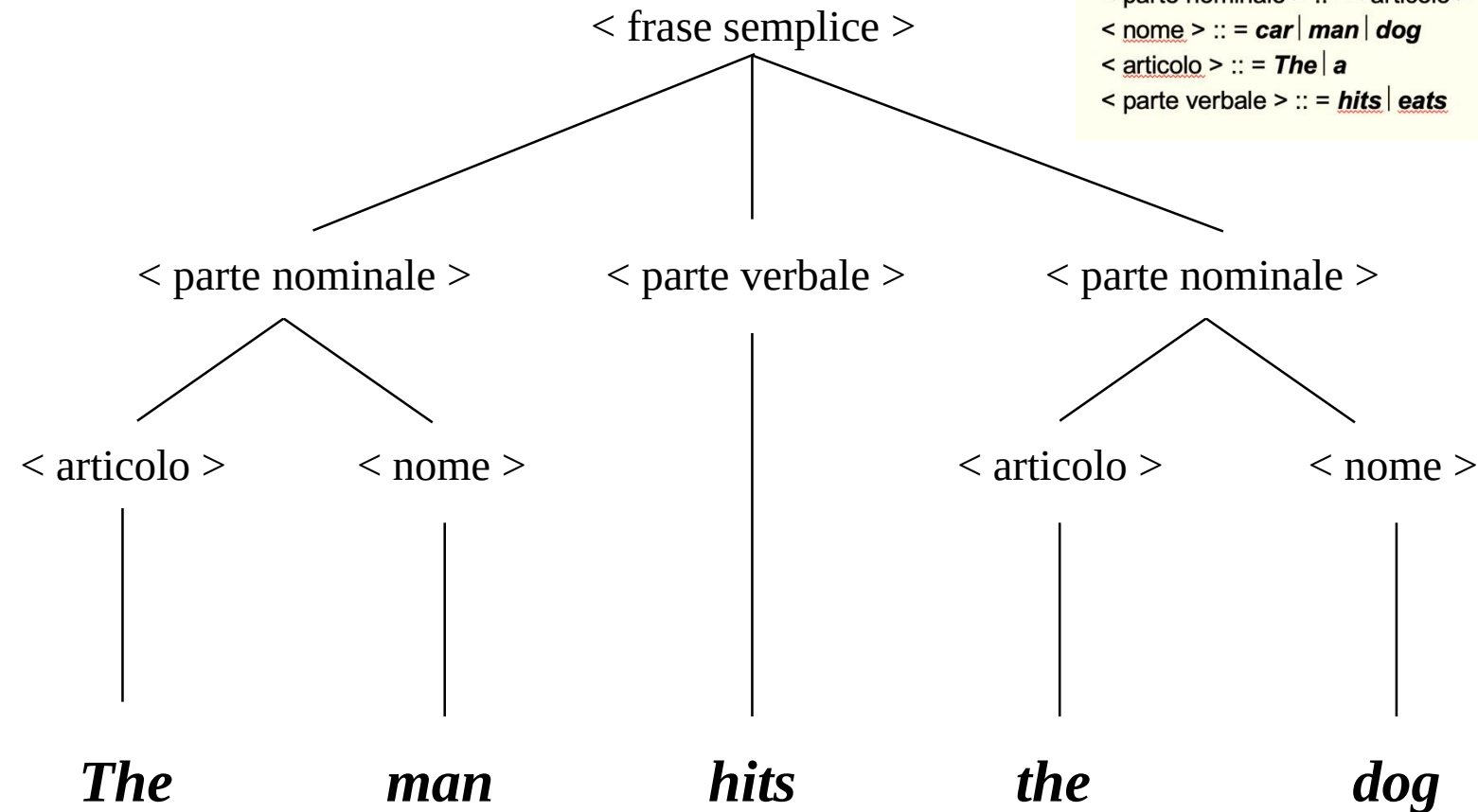


Sintassi e semantica

Provate a costruire un albero di derivazione per la frase «The Man Hits The Dog»

Albero di derivazione

< frase semplice > :: =< parte nominale > < parte verbale >
< parte nominale >
< parte nominale > :: =< articolo > < nome >
< nome > :: = *car* | *man* | *dog*
< articolo > :: = *The* | *a*
< parte verbale > :: = *hits* | *eats*



**Seguendo la stessa
sequenza di derivazione
posso ottenere altre frasi?**

< frase semplice > :: =< parte nominale > < parte verbale >
 < parte nominale >

< parte nominale > :: =< articolo > < nome >

< nome > :: = ***car*** | ***man*** | ***dog***

< articolo > :: = ***The*** | ***a***

< parte verbale > :: = ***hits*** | ***eats***



Sintassi e semantica

**Seguendo la stessa
sequenza di derivazione
posso ottenere altre frasi?**

Albero di derivazione

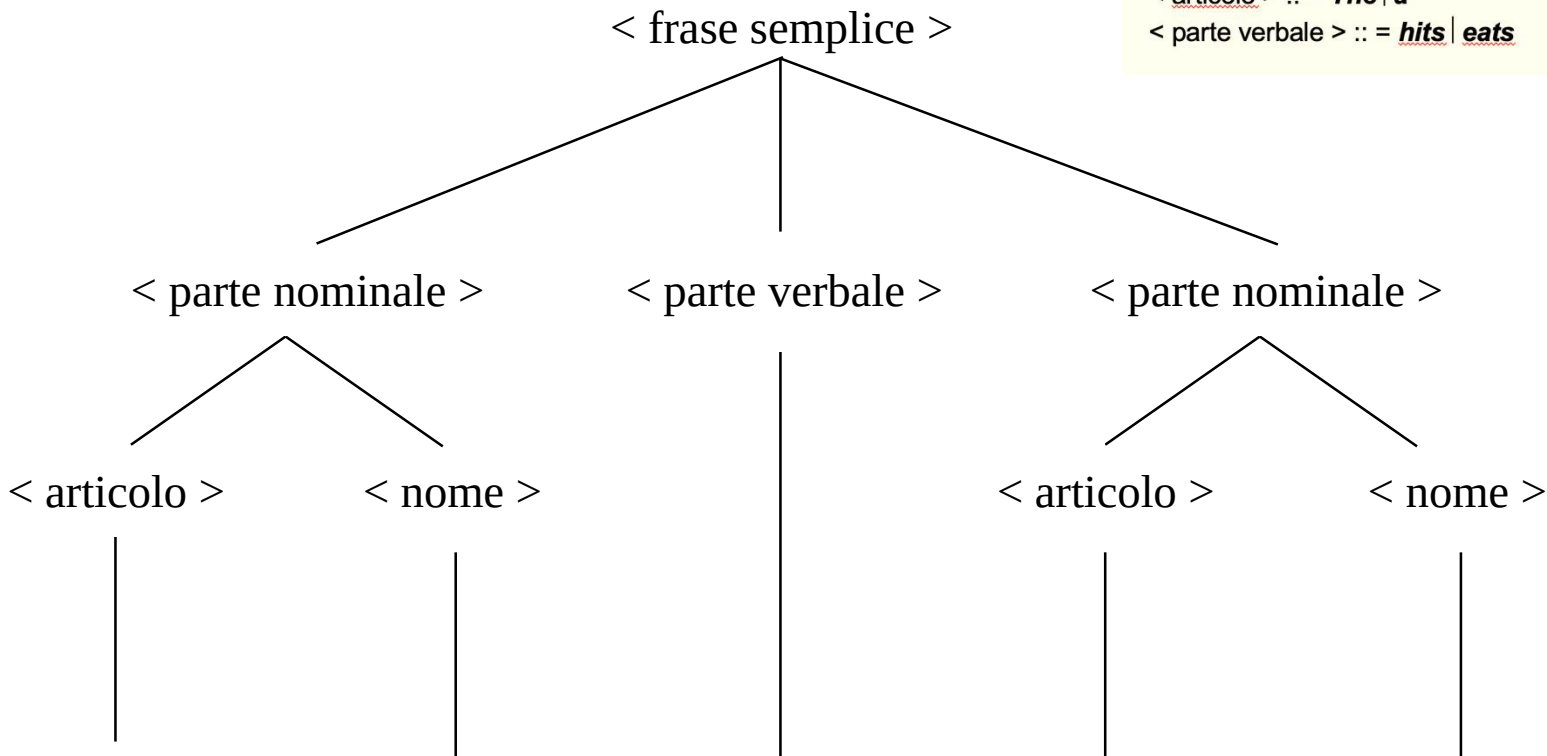
< frase semplice > :: =< parte nominale > < parte verbale >
 < parte nominale >

< parte nominale > :: =< articolo > < nome >

$$\langle \text{nome} \rangle ::= \text{car} \mid \text{man} \mid \text{dog}$$

< articolo > ::= **The** | **a**

< parte verbale > :: = hits | eats

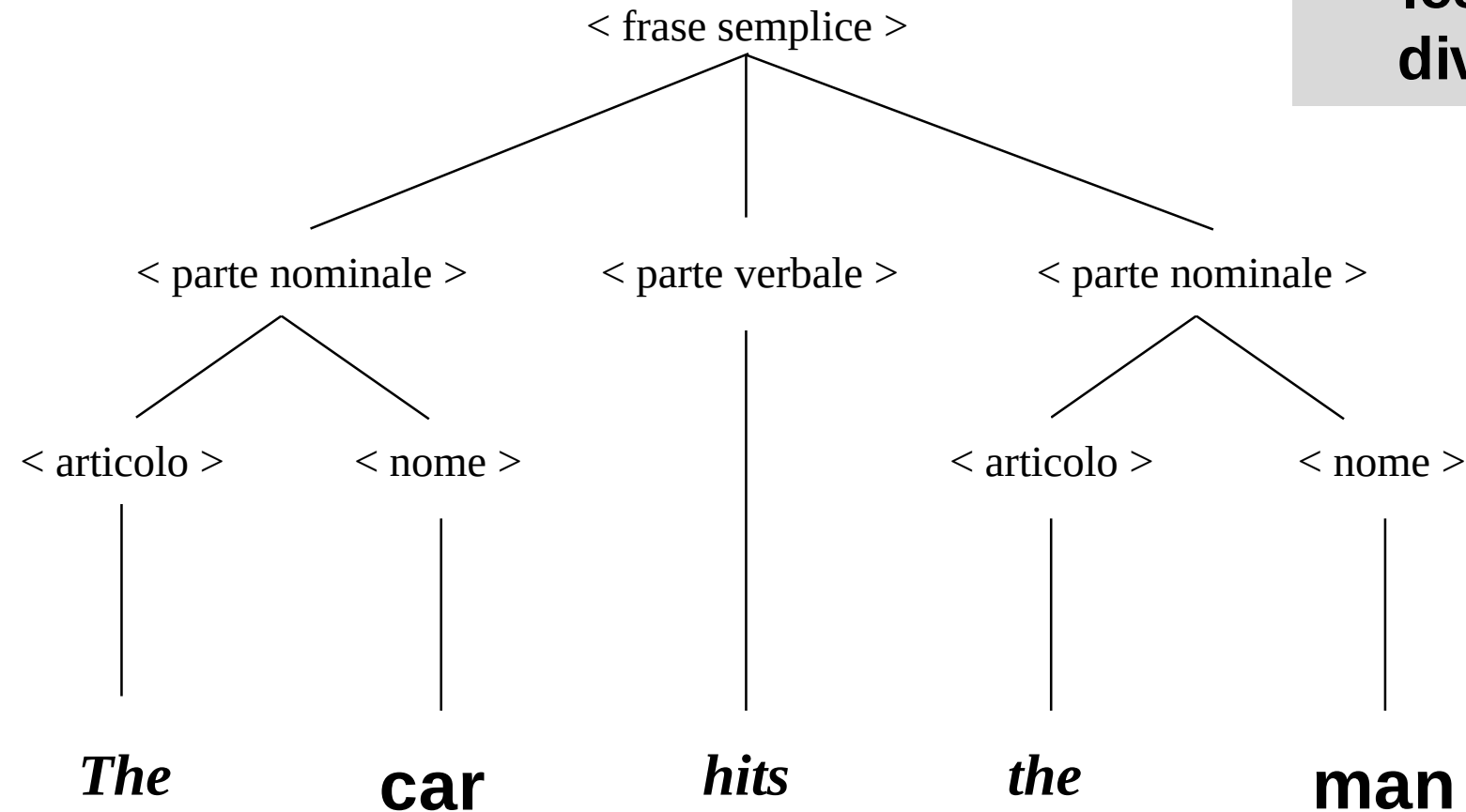


Sintassi e semantica

SI

E' sufficiente
scegliere un
lessico
diverso

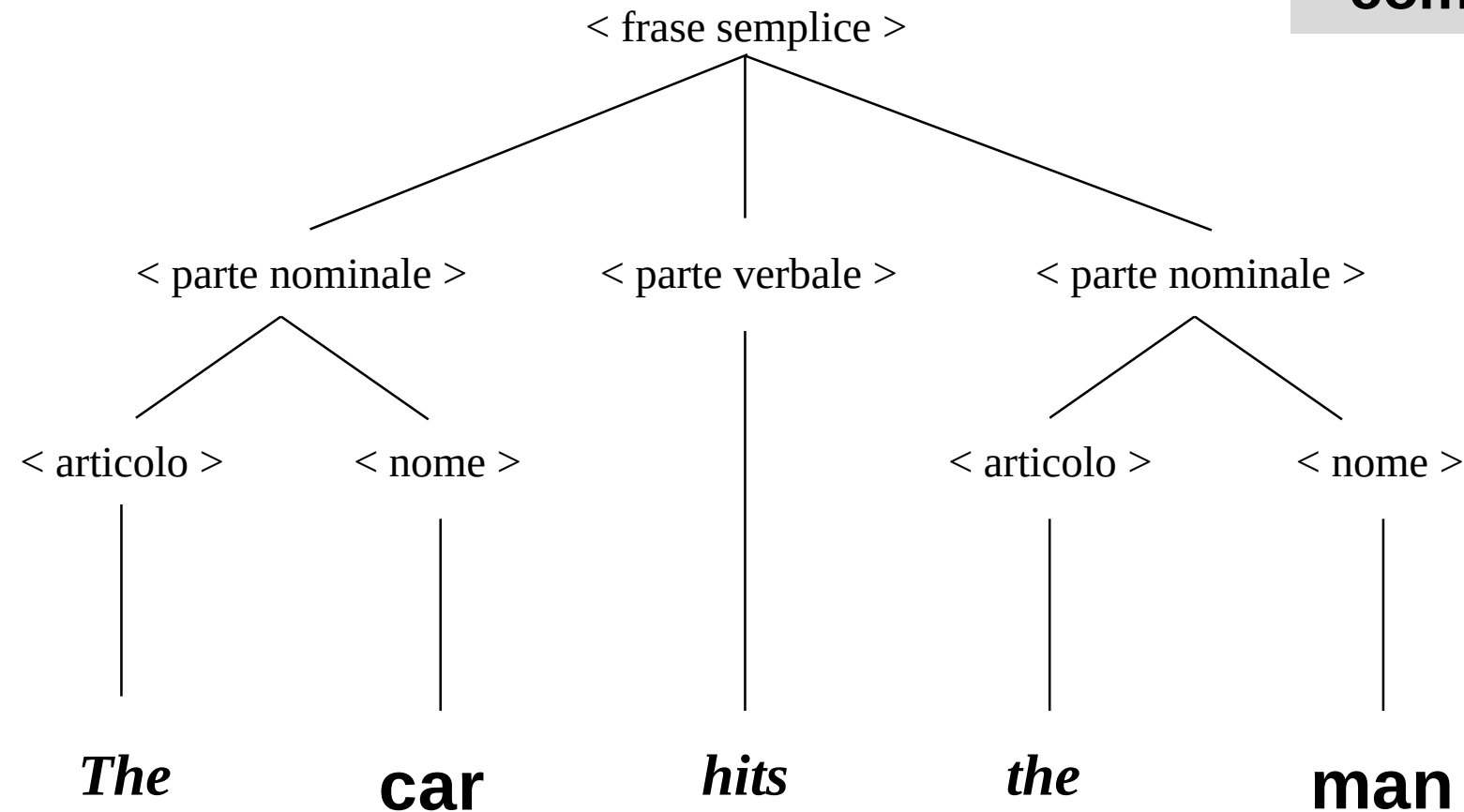
Albero di derivazione



Sintassi e semantica

Albero di derivazione

**Tutte le frasi
derivabili
sono di
senso
compiuto?**

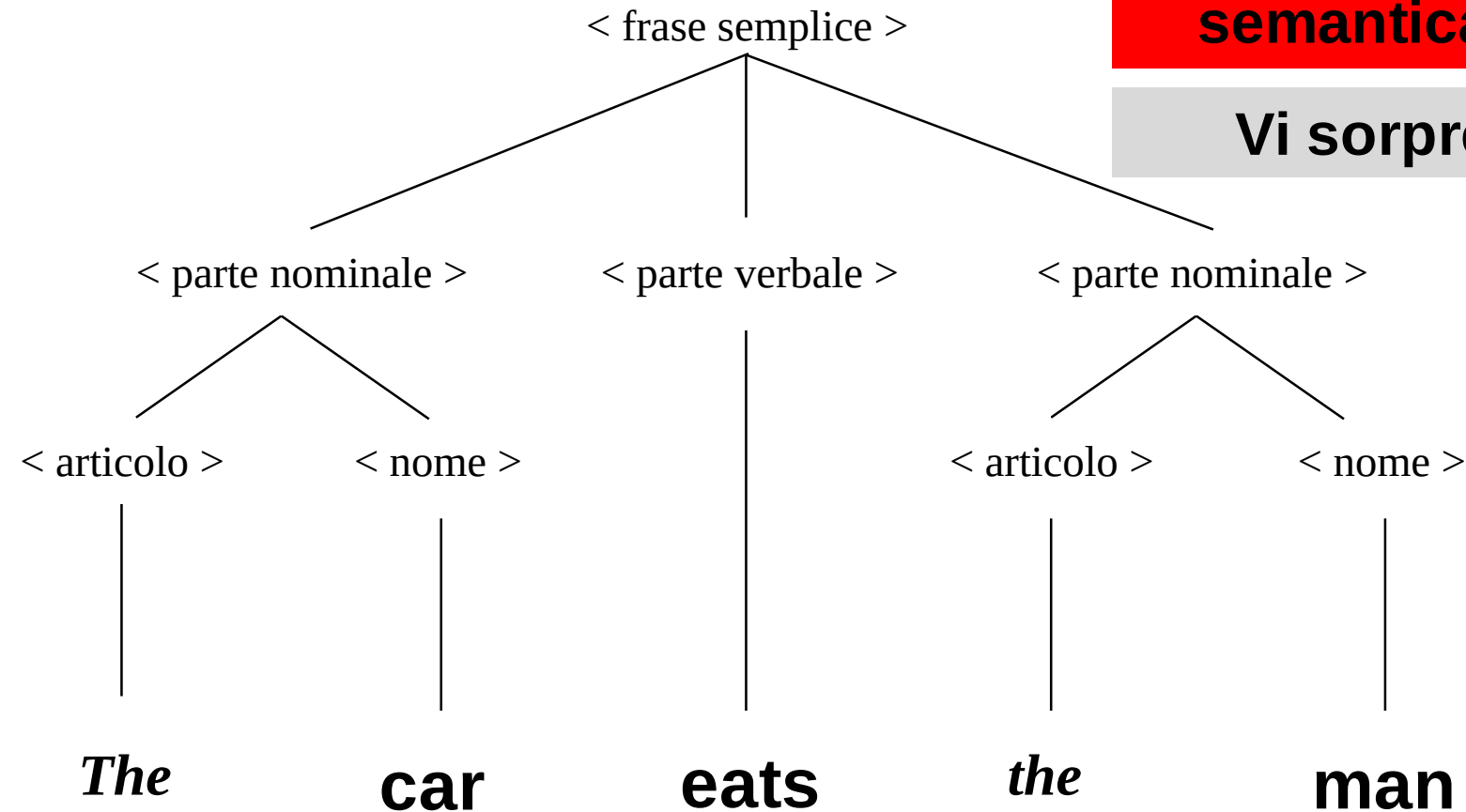


Sintassi e semantica

Albero di derivazione

**NO. E' possibile
ottenere delle
derivazioni corrette
ma che abbiano una
semantica errata**

Vi sorprende?



Sintassi e Semantica

- La correttezza è un concetto **relativo alla sintassi** di un linguaggio
 - Non abbiamo legato il concetto di **correttezza** al **significato** di una frase
 - **Per il momento tralasciamo il significato** che possiamo attribuire ad una sequenza di parole

Definizione

Un **linguaggio formale** è l'insieme di tutte le stringhe che possono essere derivate a partire dalle regole della grammatica, a prescindere dal significato!

Sintassi e Semantica

- La correttezza è un concetto **relativo alla sintassi** di un linguaggio
 - Non abbiamo legato il concetto di **correttezza** al **significato** di una frase
 - **Per il momento tralasciamo il significato** che possiamo attribuire ad una sequenza di parole
- **L'analogia regge anche per i linguaggi di programmazione**, infatti possono esistere dei programmi corretti (cioè **compilati correttamente**) ma che non fanno ciò che ci aspettiamo (**semantica errata**)

Sintassi e Semantica

- Qualche parola sulla semantica
 - Idealmente, ogni frase in un linguaggio dovrebbe essere corretta sia **semanticamente** (avere cioè il corretto significato) sia **sintatticamente** (avere la corretta struttura grammaticale).
 - In linguaggio naturale possiamo di solito comunicare anche attraverso **frasi mal strutturate e incomplete**
 - Nell'italiano parlato, le frasi sono spesso sintatticamente sbagliate, ciononostante convogliano la semantica desiderata.

Sintassi e Semantica

- In linguaggio naturale possiamo di solito comunicare anche attraverso **frasi mal strutturate e incomplete**
 - Nell'italiano parlato, le frasi sono spesso sintatticamente sbagliate, ciononostante convogliano la semantica desiderata.
- Nella programmazione questo non è possibile.
 - **Non si può prescindere dalla correttezza sintattica**
 - Significato associato alle frasi corrette
 - i nostri programmi devono **aderire rigorosamente a regole stringenti** e anche differenze minori vengono rigettate come errate.

Vi sorprende?



Questa differenza non deve sorprendere, perché i linguaggi naturali in linea di massima non sono linguaggi formali. Però, alcuni aspetti del linguaggio (es., la sintassi) **seguono sistemi formali di regole**.

Linguaggi Formali – un primo esempio

- Proviamo a costruire la nostra prima grammatica?



Linguaggi Formali – un primo esempio

- Proviamo a costruire la nostra prima grammatica per un linguaggio formale.
- Un linguaggio formale è **il linguaggio delle parentesi ben formate** che comprende tutte le stringhe di parentesi aperte e chiuse bilanciate correttamente.

Linguaggi Formali – un primo esempio

- Proviamo a costruire la nostra prima grammatica per un linguaggio formale.
- Un linguaggio formale è **il linguaggio delle parentesi ben formate** che comprende tutte le stringhe di parentesi aperte e chiuse bilanciate correttamente.

Esempio:

- () è ben formata;
- (()) è ben formata;
- (()) non è ben formata.

- Questo linguaggio è fondamentale in informatica come notazione per contrassegnare il “raggio d’azione” nelle espressioni matematiche e nei linguaggi di programmazione

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - ()  costruiamo casi più complessi a partire dal più semplice esempio

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - $()$  parentesi semplice
 - $()()$
 - $(())$
 - $((()))$
 - $()()(())$

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - $()$  parentesi semplice
 - $()()$  concatenazione
 - $(())$
 - $((()))$
 - $()()(())$

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - $()$  parentesi semplice
 - $()()$  concatenazione
 - $(())$  ricorsione
 - $((()))$
 - $()()(())$

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - $()$  parentesi semplice
 - $()()$  concatenazione
 - $(())$  ricorsione
 - $((()))$  ricorsione
 - $()()(())$

Linguaggi Formali – un primo esempio

■ Definizione

- **Quando una coppia di parentesi è «ben formata»?**
- Partiamo da alcuni esempi e proviamo a ricavare delle regole generali
- Esempi di parentesi ben formate:
 - $()$  parentesi semplice
 - $()()$  concatenazione
 - $(())$  ricorsione
 - $((()))$  ricorsione
 - $()()(())$  concatenazione e ricorsione
- **A partire dalle caratteristiche, proponiamo delle regole**

Linguaggi Formali – un primo esempio

■ Esempi di parentesi ben formate:

- $()$ parentesi semplice
- $(())$ ricorsione
- $((()))$ ricorsione
- $()()$ concatenazione
- $()()(())$ ricorsione + concatenazione

■ Possibili regole

- Costruiamo una prima regola a partire dalla più semplice 'parola' del linguaggio (in questo caso, la più semplice parentesi)

Linguaggi Formali – un primo esempio

■ Esempi di parentesi ben formate:

- $()$ parentesi semplice
- $(())$ ricorsione
- $((()))$ ricorsione
- $()()$ concatenazione
- $()()(())$ ricorsione + concatenazione

■ Possibili regole

1. La stringa $()$ è ben formata;

Linguaggi Formali – un primo esempio

■ Esempi di parentesi ben formate:

- $()$ parentesi semplice
- $(())$ ricorsione
- $((()))$ ricorsione
- $() ()$ concatenazione
- $() () (())$ ricorsione + concatenazione

■ Possibili regole

1. La stringa $()$ è ben formata;

Come possiamo costruire **una stringa più complessa** a partire da una stringa ben formata?

Linguaggi Formali – un primo esempio

■ Esempi di parentesi ben formate:

- $()$ parentesi semplice
- $(())$ ricorsione
- $((()))$ ricorsione
- $()()$ concatenazione
- $()()(())$ ricorsione + concatenazione

■ Possibili regole

1. La stringa $()$ è ben formata;

Come possiamo costruire una **stringa più complessa** a partire da una stringa ben formata?

1. Opzione a: con la ricorsione
2. Opzione b: concatenando

Linguaggi Formali – un primo esempio

■ Esempi di parentesi ben formate:

- $()$ parentesi semplice
- $(())$ ricorsione
- $((()))$ ricorsione
- $()()$ concatenazione
- $()()(())$ ricorsione + concatenazione

■ Possibili regole

1. La stringa $()$ è ben formata;
2. se la stringa di simboli A è ben formata, allora lo è anche (A) ;
3. se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .

Linguaggi Formali – un primo esempio

■ Definizione

1. La stringa $()$ è ben formata;
2. se la stringa di simboli A è ben formata, allora lo è anche (A) ;
3. se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .

- A partire da questa definizione, possiamo generare **un sistema di regole come quelli già visti attraverso cui costruire** l'insieme delle stringhe corrette di parentesi ben formate:

Linguaggi Formali – un primo esempio

Definizione	Regola
La stringa $()$ è ben formata	
se la stringa di simboli A è ben formata, allora lo è anche (A)	
se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .	

Linguaggi Formali – un primo esempio

Definizione	Regola
La stringa $()$ è ben formata	$S \rightarrow ()$
se la stringa di simboli A è ben formata, allora lo è anche (A)	
se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .	

Linguaggi Formali – un primo esempio

Definizione	Regola
La stringa $()$ è ben formata	$S \rightarrow ()$
se la stringa di simboli A è ben formata, allora lo è anche (A)	$S \rightarrow (S)$
se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .	

Linguaggi Formali – un primo esempio

Definizione	Regola
La stringa $()$ è ben formata	$S \rightarrow ()$
se la stringa di simboli A è ben formata, allora lo è anche (A)	$S \rightarrow (S)$
se le stringhe A e B sono ben formate, allora lo è anche la stringa AB .	$S \rightarrow SS$

Ogni elemento della definizione è
mappato con una regola.

Linguaggio delle parentesi ben formate

- Grammatica delle parentesi ben formate:
 - (1) $S \rightarrow ()$
 - (2) $S \rightarrow (S)$
 - (3) $S \rightarrow SS$

- Rispetto alla notazione precedente ci sono due differenze
 - Il simbolo $::=$ è sostituito da $\rightarrow$
 - I simboli non terminali sono rappresentati senza parentesi angolari ma in maiuscolo
 - **Manteniamo da ora in poi queste convenzioni**

Linguaggio delle parentesi ben formate

- Grammatica delle parentesi ben formate:
 - (1) $S \rightarrow ()$
 - (2) $S \rightarrow (S)$
 - (3) $S \rightarrow SS$
- E' importante notare che nella parte sinistra utilizziamo il simbolo 'S' (anche detto simbolo di partenza). E' il simbolo che si utilizza per cominciare una qualsiasi derivazione, equivalente alle varie 'Regola #1' viste negli esempi precedenti

Linguaggio delle parentesi ben formate

- Grammatica delle parentesi ben formate:
 - (1) $S \rightarrow ()$
 - (2) $S \rightarrow (S)$
 - (3) $S \rightarrow SS$
- Una ulteriore differenza si può apprezzare nella regola (3). Sebbene nella definizione fossero utilizzati due simboli distinti (A e B), nella regola si utilizza un unico simbolo. **Non c'è esigenza di differenziare**, perché da ogni simbolo S nella Regola (3) si svilupperà in modo indipendente una derivazione.

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$

Obiettivo: trovare una sequenza di regole che ci permetta di ottenere la stringa valida a partire dalla grammatica che abbiamo definito

$$\begin{array}{ll} (1) & S \rightarrow () \\ (2) & S \rightarrow (S) \\ (3) & S \rightarrow SS \end{array}$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:
(3)

$$S \xRightarrow{(3)} SS$$

(1) $S \rightarrow ()$
(2) $S \rightarrow (S)$
(3) $S \rightarrow SS$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:
 $(3) \rightarrow (2)$

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)$$

(1) $S \rightarrow ()$
(2) $S \rightarrow (S)$
(3) $S \rightarrow SS$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:

$$(3) \rightarrow (2) \rightarrow (1)$$

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)S \xRightarrow{(1)} (())$$

$$\begin{array}{ll} (1) & S \rightarrow () \\ (2) & S \rightarrow (S) \\ (3) & S \rightarrow SS \end{array}$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:

$$(3) \rightarrow (2) \rightarrow (1) \rightarrow (2)$$

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)S \xRightarrow{(1)} (())S \xRightarrow{(2)} (())(S)$$

$$(1) \quad S \rightarrow ()$$

$$(2) \quad S \rightarrow (S)$$

$$(3) \quad S \rightarrow SS$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:

$$(3) \rightarrow (2) \rightarrow (1) \rightarrow (2) \rightarrow (3)$$

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)S \xRightarrow{(1)} (())S \xRightarrow{(2)} (())(S) \xRightarrow{(3)} (())(SS)$$

$$(1) \quad S \rightarrow ()$$

$$(2) \quad S \rightarrow (S)$$

$$(3) \quad S \rightarrow SS$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:

$$(3) \rightarrow (2) \rightarrow (1) \rightarrow (2) \rightarrow (3) \rightarrow (1) \rightarrow (1)$$

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)S \xRightarrow{(1)} (())S \xRightarrow{(2)} (())(S) \xRightarrow{(3)} (())(SS) \xRightarrow{(1)} (())(())S \xRightarrow{(1)} (())(())(())$$

$$(1) \quad S \rightarrow ()$$

$$(2) \quad S \rightarrow (S)$$

$$(3) \quad S \rightarrow SS$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - La corrispondente sequenza di applicazione delle produzioni è:

$$(3) \rightarrow (2) \rightarrow (1) \rightarrow (2) \rightarrow (3) \rightarrow (1) \rightarrow (1)$$

- Una sequenza di applicazioni di regole di produzione prende il nome di *derivazione*.

$$S \xRightarrow{(3)} SS \xRightarrow{(2)} (S)S \xRightarrow{(1)} (())S \xRightarrow{(2)} (())(S) \xRightarrow{(3)} (())(SS) \xRightarrow{(1)} (())(())S \xRightarrow{(1)} (())(())(())$$

$$\begin{array}{ll} (1) & S \rightarrow () \\ (2) & S \rightarrow (S) \\ (3) & S \rightarrow SS \end{array}$$

Linguaggio delle parentesi ben formate

- Esempio: Generazione di $(())(())(())$
 - Esiste un altro modo?

Linguaggio delle parentesi ben formate

■ Esempio: Generazione di $(())(()())$

□ Secondo modo:

$$S \underset{(3)}{\Rightarrow} SS \underset{(2)}{\Rightarrow} S(S) \underset{(3)}{\Rightarrow} S(SS) \underset{(1)}{\Rightarrow} S(()S) \underset{(1)}{\Rightarrow} S(()()) \underset{(2)}{\Rightarrow} (S)(()()) \underset{(1)}{\Rightarrow} (())(()())$$

□ La corrispondente sequenza di applicazione delle produzioni è:

$$|(3) \rightarrow (2) \rightarrow (3) \rightarrow (1) \rightarrow (1) \rightarrow (2) \rightarrow (1)$$

Come detto in precedenza, **non c'è un ordine predefinito** nella sequenza di applicazioni delle regole.

Notazione

- Nei due esempi precedenti abbiamo fatto uso della notazione

$$\alpha \underset{(n)}{\Rightarrow} \beta$$

- L'interpretazione è la seguente: “da α si produce direttamente β per effetto dell'applicazione della regola di riscrittura n ”. Ad esempio

$$SS \underset{(2)}{\Rightarrow} (S)S$$

- si legge: “SS produce direttamente (S)S per effetto dell'applicazione della regola di riscrittura (2)”.

Notazione

- **Una sequenza di applicazione di regole** viene generalmente riassunta con il simbolo $*$ in questo modo

$$S \xRightarrow{*} (())(())$$

che leggiamo come “S produce $(())(())$ ”

Significa che da S posso derivare $(())(())$
applicando una **sequenza di regole** (di una qualsiasi lunghezza)

Linguaggi Formali e Compilatori

■ Un piccolo recap conclusivo

- Nei linguaggi formali, le grammatiche possono essere utilizzate **per definire le regole** che determinano i meccanismi di **costruzione di stringhe corrette;**
 - Nei linguaggi di programmazione → stringhe corrette = programmi corretti
- Dato una grammatica e data una stringa, posso costruire un algoritmo che mi dica se la stringa può essere corretta per quella grammatica?

Linguaggi Formali e Compilatori

■ Un piccolo recap conclusivo

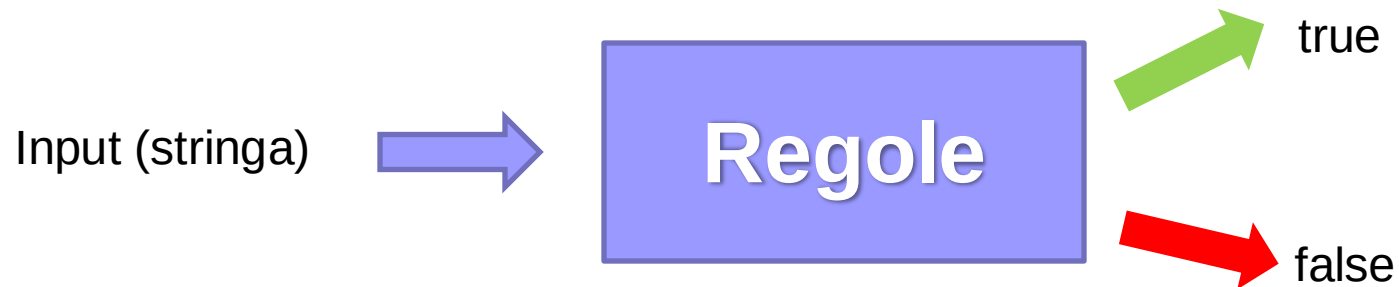
- Nei linguaggi formali, le grammatiche possono essere utilizzate **per definire le regole** che determinano i meccanismi di **costruzione di stringhe corrette;**
 - Nei linguaggi di programmazione → stringhe corrette = programmi corretti
- Dato una grammatica e data una stringa, posso costruire un algoritmo che mi dica se la stringa può essere corretta per quella grammatica?

SI

Linguaggi formali e Compilatori

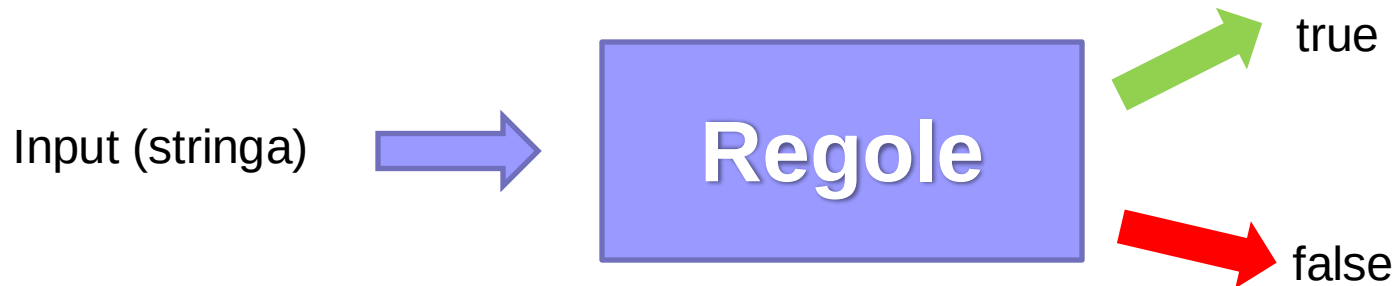
Regole

Linguaggi formali e Compilatori

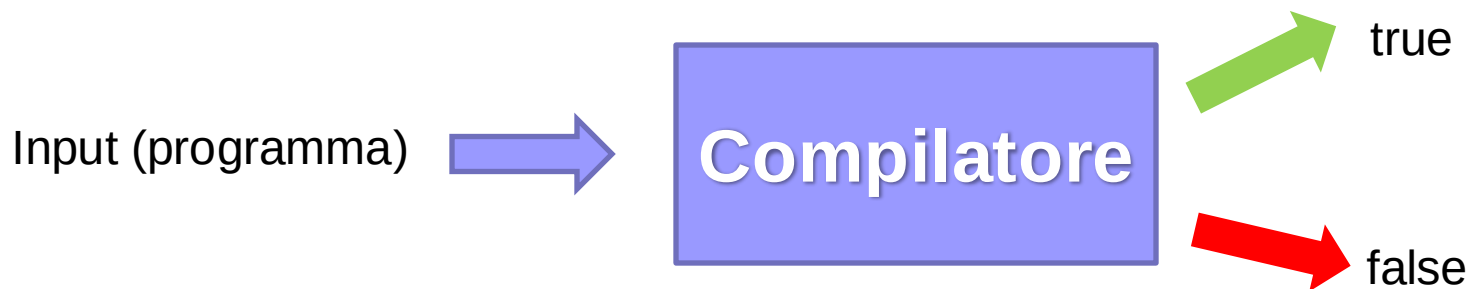


Cosa ci ricorda?

Linguaggi formali e Compilatori

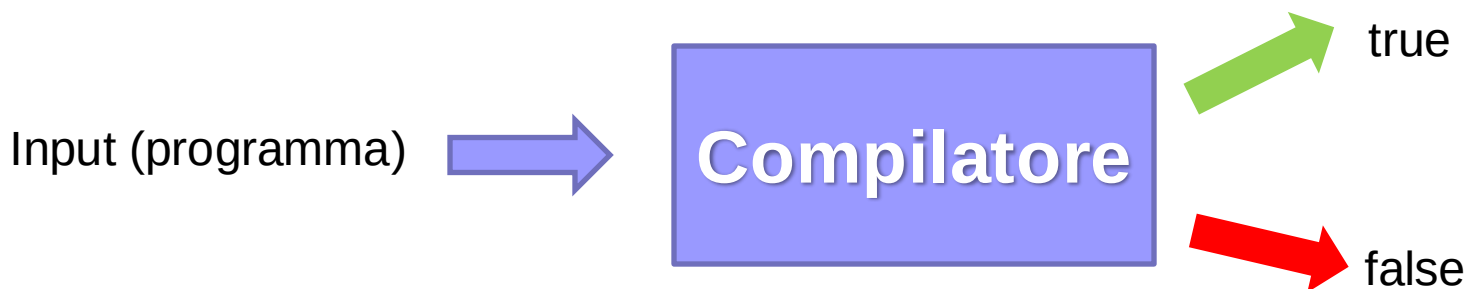


Cosa ci ricorda?



Linguaggi Formali e Compilatori

- Nell'ambito dei linguaggi formali, se esiste una grammatica che permette di generare un linguaggio allora **esiste anche uno strumento che ci permette di stabilire se una stringa è stata costruita in modo corretto!**
 - **Equivalente ai compilatori** nei linguaggi di programmazione



Linguaggi Formali e Compilatori

- Nell'ambito dei linguaggi formali, se esiste una grammatica che permette di generare un linguaggio allora **esiste anche uno strumento che ci permette di stabilire se una stringa è stata costruita in modo corretto!**
 - **Equivalente ai compilatori** nei linguaggi di programmazione
- **Linguaggio di Programmazione = Linguaggio Formale**
 - Anche i linguaggi di programmazione hanno un alfabeto e un lessico
 - La sintassi dei linguaggi di programmazione può essere ricondotta a un insieme di regole

Take-home Messages (3)

- Concetti di base della teoria dei linguaggi formali
 - **Linguaggio formale** = insieme di stringhe
 - **Grammatica** = insieme di regole che consentono di derivare stringhe di un linguaggio
 - **Stringa corretta** = stringa derivabile, ossia costruita applicando le regole della grammatica
- **Un programma è una stringa**
 - Programma corretto = derivabile con le regole della relativa grammatica
 - **Un compilatore verifica la correttezza sintattica** di una stringa (il programma sorgente dato in input)

Riferimenti

- Semeraro, G., Elementi di Teoria dei Linguaggi Formali, ilmiolibro.it, 2017
(<http://ilmiolibro.kataweb.it/libro/informatica-e-internet/317883/elementi-di-teoria-dei-linguaggi-formali/>).
 - Capitolo 1

Domande?



Linguaggi di Programmazione
docente: Cataldo Musto
cataldo.musto@uniba.it